UNIVERSITY
INSTITUTE
OF LISBON

Measuring the Coordination Interfaces of Demand-Driven Edge Services

Pedro Ojo Oriakhi

Master's in Telecommunications and Computer Engineering

Supervisors:

Ph.D. Professor José Moura, Assistant Professor,
Iscte - Instituto Universitário de Lisboa

Ph.D. Professor Pedro Ramos, Assistant Professor,
Iscte - Instituto Universitário de Lisboa

Month, Year

TECHNOLOGY
AND ARCHITECTURE

Department of Information Science and Technology

Measuring the Coordination Interfaces of Demand-Driven Edge Services

Pedro Ojo Oriakhi

Master's in Telecommunications and Computer Engineering

Supervisors:

Ph.D. Professor José Moura, Assistant Professor,
Iscte - Instituto Universitário de Lisboa

Ph.D. Professor Pedro Ramos, Assistant Professor,
Iscte - Instituto Universitário de Lisboa

Month, Year

Write here your dedication.

Acknowledgments

Write here your acknowledgements and, if applicable, any grants or sources of funding.

Resumo

O aumento de capacidade num serviço não garante, por si só, maior disponibilidade aos utilizadores em espera. A nova capacidade pode chegar demasiado tarde, ser atribuída a uma componente do serviço que não está sob pressão ou permanecer inativa ou receber tráfego antes de estar apta a responder. Embora estas falhas sejam referidas na literatura, nenhum dos trabalhos revistos caracteriza o comportamento de cada etapa sob diferentes condições de funcionamento. Esta dissertação varia cada etapa isoladamente, num ambiente experimental emulado com duas redes, um controlador de rede definida por software, servidores em contentores e uma base de dados de documentos, avaliando o desempenho da perspetiva do utilizador. Os resultados mostram que a perda parcial de informação sobre a carga atrasa significativamente a disponibilização de capacidade útil e aumenta os tempos de espera. A informação completa, mas tardia, só prejudica o desempenho quando os servidores não dispõem de capacidade de reserva. A atribuição de recursos à componente sob pressão é determinante para beneficiar os utilizadores, podendo uma atribuição incorreta deixar a componente sob pressão pior do que a ausência de capacidade adicional. A validação dos anúncios de prontidão dos servidores revela que agir sem verificação é mais rápido, mas a perda do anúncio impede a admissão de tráfego, enquanto um anúncio prematuro pode encaminhar pedidos para servidores ainda indisponíveis. As conclusões apontam uma propriedade decisiva em cada etapa: a informação disponível, a componente a que se atribui capacidade e a verificação do anúncio de prontidão.

Palavras-Chave: *Computação na periferia; redes definidas por software; escalonamento automático; telemetria; MongoDB; admissão por prontidão*

Abstract

More capacity in a service does not by itself give the waiting users greater availability. New capacity can arrive too late to matter, it can go to a part that was not under strain, and it can sit unused or receive traffic before it can answer. Each failure appears in the literature, but none of the reviewed studies characterises the behaviour of each step under different operating conditions. This dissertation varies one step at a time on an emulated platform with two network domains, built from a software-defined network controller, containerised servers and a document database, watching it as a user would. When part of the load information that drives the adjustment goes missing, new capacity takes much longer to become useful and users wait far longer, whereas complete but late information hurts only when the running servers have no spare capacity. Adding capacity to the part of the service under strain decides whether users gain anything, and adding it elsewhere can leave the pressed component worse off than adding nothing. When a newly started server announces that it is ready, acting on that message unchecked is the fastest option and the most fragile as it saves time while the platform is healthy, admits nothing once the message is lost, and lets traffic reach a server that cannot yet serve it. The conclusions point to one deciding property at each step: the information available, the tier that receives the capacity, and the checking of the readiness message.

Keywords: *Edge computing; software-defined networking; auto-scaling; telemetry; MongoDB; readiness admission*

Contents

Acknowledgments iii

Resumo v

Abstract vii

List of Figures xi

List of Tables xiii

List of Acronyms xvii

Chapter 1. Introduction 1
1.1. Context, Motivation, and Problem Statement 1
1.2. Objectives 3
1.3. Research Questions 4
1.4. Research Methodology 4
1.5. Main Contributions of this Dissertation 5
1.6. Dissertation Structure 6

Chapter 2. Background and Related Work 7
2.1. Review Methodology 7
2.2. Platform Technology Background 7
2.3. Telemetry 8
2.4. Auto Scaling 9
2.5. Readiness Admission in SDN Load Balancing 10
2.6. SDN-Based Orchestration Platforms 11
2.7. Synthesis and Research Gaps 12

Chapter 3. Architecture and Design 13
3.1. Design Requirements 13
3.2. Architecture 14
3.2.1. Two-Domain Organisation 15
3.2.2. Virtual IP Data Paths 16
3.2.3. Control and Telemetry Paths 16
3.3. Distributed Elastic Allocation of Resources 16
3.3.1. Scaling Policy 17
3.3.2. Compute 19

3.3.3. Storage 20
3.4. Telemetry Delivery and Scaling 20
3.5. Control Workflow 21

Chapter 4. Implementation 23
4.1. Experimental Infrastructure 23
4.2. Data Path 24
4.3. Distributed Elastic Allocation of Resources 24
4.3.1. Compute 24
4.3.2. Storage 25
4.4. Telemetry Delivery and Scaling 26
4.5. Control Workflow 27

Chapter 5. Experimental Results 29
5.1. Experimental Setup 29
5.1.1. Testbed and Instrumentation 29
5.1.2. Demand Model and Campaign Workloads 29
5.1.3. Metrics and Outcome Definitions 30
5.1.4. Campaign Designs and Analysis 31
5.2. Telemetry Modes (RQ1) 32
5.3. Bottleneck-Aware Scaling Action (RQ2) 34
5.4. Readiness Admission Modes (RQ3) 38
5.5. Discussion: Synthesis Across Interfaces 43

Chapter 6. Conclusions and Future Work 47
6.1. Conclusions 47
6.2. Research Contributions 48
6.3. Limitations 48
6.4. Future Research Directions 49

References 51

Appendix A. Campaign Configuration Values 55

Appendix B. Implementation Validation 61

Appendix C. Deployment and Delivery Configuration 63

Appendix D. Review Corpus 65

List of Figures

Figure 1.1 Global internet traffic between 2020 and 2025. Source: Statista, based on ITU data [3]. 1

Figure 1.2 The demand-to-capacity chain: from a demand shift to usable capacity, with the three interfaces studied in this dissertation highlighted. 3

Figure 1.3 Design Science Research Methodology 4

Figure 3.1 Layered organisation of the platform in one network domain: the controller of the control layer consumes the telemetry summaries, decides how the traffic is routed, and issues the capacity changes to the resources. The data plane layer carries the client requests and the storage traffic, and the resource layer holds the edge servers, the storage members, and the telemetry aggregator. 15

Figure 3.2 Elastic allocation of resources: each tier is scaled on its own observation signals, with compute replicas provisioned and drained and storage members added and removed through replica-set membership, both actions issued by the controller over the infrastructure-management path. 17

Figure 3.3 The control workflow, drawn as a UML diagram. The demand-to-capacity chain runs across three contexts in the same process: the packet context, the observation context, and the action context. The contexts do not block one another and they exchange state through shared structures (the telemetry state, the action queue, the capacity state, and the routing pools). The numbered stages match the chain in Figure 1.2 and show the provisioning direction. Capacity removal follows the same steps in reverse, starting with the drain. The aggregator is omitted between the observations sent by the replicas and the summary delivered to the controller. 22

Figure 5.1 Episode endpoint latency p95 by telemetry mode (served basis; per-run mean of the two domains; seven replicates per mode). Bars show the median across replicates, whiskers the min–max range, and dots the individual runs; the two highest delayed push points are the replicates where the spare capacity of the nodes already running was consumed. 34

Figure 5.2 Mixed press: processing (left), data access (middle), and completion (right) by scaling policy. Bars are medians across the seven seed-matched runs,

whiskers the min–max range, and dots the individual runs. Policy colours follow the chapter's fixed order. 36

Figure 5.3 Data-access press: processing (left), data access (middle, logarithmic axis), and completion (right) by scaling policy. Bars are medians across the seven seed-matched runs, whiskers the min–max range, and dots the individual runs. 37

Figure 5.4 Relief and execution, by scaling policy: mixed press storage utilisation (a) and members serving (b); data-access press storage utilisation (c) and members serving (d). Bars are medians across the runs, whiskers the min–max range, and dots the individual runs. 38

Figure 5.5 Admission-chain intervals under an intact message, for the four admission modes. (a) Provisioning to first successful response per backend (medians 4.1, 3.1, 10.0 and 4.0 s for backends that bind in under a second; 13.7, 13.7 and 19.8 s for the backends that met the stall at bind, none of them under the confirmation check). (b) Scaling decision to usable capacity per run (higher-rate set, six runs per mechanism). Dots are individual backends (a) and runs (b), bars and marks are medians, and whiskers span the min–max range. 40

Figure 5.6 Under a lost message with reduced headroom, only the modes that check recover. (a) Slow share (completed or timed-out requests that took longer than 1 s), median across the seven seed-matched blocks per 10 s bin; the shaded band marks the first 150 s of the plateau. (b) Admitted backends per run (dots, one per block; bars, medians; whiskers, the min–max range). Backend notification admitted no backend and abandoned 32–35 candidates per run, while the other three modes admit through their own checks (medians 5, 5 and 6). The gated contrasts use each mode's assessable blocks (Table 5.9). 42

Figure A.1 Phases of a run, shown for the telemetry campaign. The phase that applies the demand shift under test is the demand episode, and the outcomes are read over it. The surrounding phases have fixed roles: the baseline window establishes the healthy pre-shift operating point, the recovery gap lets the platform settle, the demand drop is the release window in which the episode's capacity is scaled down, and the idle tail, where a campaign uses one, lets the release complete. Durations, intensities, and mixes differ per campaign and are listed in Table 5.1. 55

List of Tables

Table 2.1 The three interfaces of the demand-to-capacity chain and the questions they leave open. 12

Table 5.1 Campaign workloads. Each campaign defines its own phase sequence, run in the order shown. The demand episode is the phase that applies the demand shift under test. 30

Table 5.2 Design of the telemetry campaign (RQ1). 32

Table 5.3 Telemetry modes: what the controller received and what users experienced. Delivered and information age are means over the seven replicates, with the observed range in parentheses; the other columns are medians. Information age is the time from the closing of the summary a decision used to the decision itself, and the metric definitions are in Section 5.1.3. No timeouts occurred in any mode. 33

Table 5.4 Pre-registered primary contrasts on the episode endpoint latency p95 (served basis, per-run mean of the two domains). Cliff's δ is given with its 95 % bootstrap interval where the runs overlap, and the exact p-value only where the runs overlap, since complete separation at seven runs per mode has the floor stated in Section 5.1.4. The block count is how many seed-matched blocks followed the predicted order. 33

Table 5.5 Design of the scaling action campaign (RQ2). 35

Table 5.6 Scaling policies by demand episode: the two request-path components, completion, storage utilisation, and the actions taken. Processing and data access are the components defined in Section 3.2.2, completion is completed requests over offered requests during the episode, and storage utilisation is the tier average over the episode. Values are medians across the seven seed-matched runs per policy. Compute scale-ups are per domain and run. Storage admissions count the activation of a prepared reserve and the preparation of its replacement, per domain and run. They are admission events, not concurrent members, and the concurrent count stays within the 6/6 cap. The figures show the per-run values. 36

Table 5.7 Design of the readiness admission campaign (RQ3). 39

Table 5.8 Admission-chain intervals and healthy-regime outcomes (intact message; backend notification versus periodic polling). Medians over each row's

population: the readiness→admission row covers the recorded admission-timing set (12 runs, 47 backends), provisioning→first success pools both start-up classes, and scaling decision→usable capacity uses the higher-rate set. Exact Mann–Whitney p-values where the modes overlap. 40

Table 5.9 Admission modes under a lost message (fixed six-block frame, five assessable; the confirmation check covers its seven blocks). Admitted means visibility in the routing pool, with abandoned candidates in parentheses. Plateau slow is the slow share over the full plateau, plateau tail over its last 300 s, and a named block gives that block's own value. 41

Table 5.10 User-visible damage of the admission modes under a premature message at the evaluated ten-second lead (seven seed-matched pairs per mode). Damage is the change, in percentage points, in the share of offered requests that fail, time out, or exceed one second, against a matched no-fault run; medians over the usable pairs (six under backend notification, seven under the other modes). A positive reading means the fault run's share was above that of its no-fault partner. 43

Table 5.11 Synthesis of the three campaigns, classified by whether a finding held for every replicate entering the contrast (robust), depended on a condition the campaign varied or sampled (conditional), or could not be confirmed (not established). Every entry remains within its own campaign, and no quantity is combined across campaigns. 44

Table 6.1 Answers to the research questions and the evidence that reports them. The classification of each finding within its own campaign is in Table 5.11. 47

Table A.1 Container resource limits, flow timeouts, and client bounds per campaign. 56

Table A.2 Scaling-policy parameters per campaign. 57

Table A.3 Load-balancing weights per campaign. 57

Table A.4 Control-plane cost per configuration (descriptive; medians across the runs of each configuration). Delivery records count delivered telemetry summaries; action records count provisioned units; admission records count events or readiness checks. The admission rows list the direct and discovery configurations; the fallback and confirmation checks are not tabulated. 58

Table A.5 Implementation knobs of the interface experiments. 59

Table A.6 The six request kinds that make up the workload mixes, and the tier each loads. 60

Table B.1 Behaviour established before the experiment campaigns, with the evidence of each check. 61

Table C.1 Platform components as deployed. 63

Table C.2 The four telemetry modes and their implemented sources. 63

Table D.1 The 27 interface works, by interface. The middle column merges what a work varies with what it measures, and a work appears under its first interface. 65

List of Acronyms

SDN: Software-Defined Networking
VIP: Virtual IP
DSRM: Design Science Research Methodology

CHAPTER 1

Introduction

Edge web services that encompass monitoring, auto-scaling and routing must coordinate these functions across the boundaries between them. How well such a service absorbs a change in demand therefore depends not only on what each function does individually, but also on the interfaces between them. Coordination across those interfaces can fall short, and where it does, orchestration efficiency and reliability can degrade and may reduce the quality of service users experience [1, 2].

The remainder of this chapter is organised as follows. Section 1.1 presents the context, motivation and problem statement; Section 1.2 states the objectives; Section 1.3 lists the research questions; Section 1.4 details the methodology used to develop the thesis; Section 1.5 summarises the main contributions; and Section 1.6 outlines the rest of the dissertation.

1.1. Context, Motivation, and Problem Statement

In 2025, global internet traffic across fixed and mobile networks exceeded 7 000 exabytes[1], more than doubling since 2020 [3], as shown in Figure 1.1.

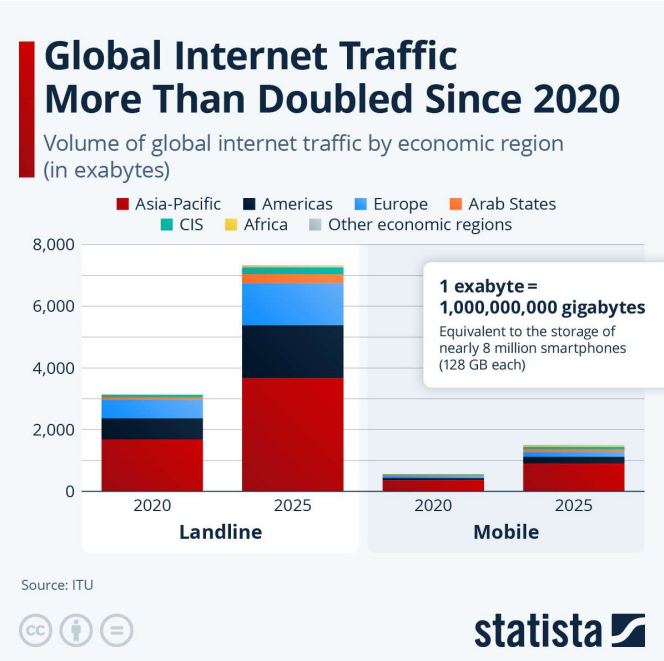


FIGURE 1.1. Global internet traffic between 2020 and 2025. Source: Statista, based on ITU data [3].

[1] One exabyte equals 10^9 gigabytes.

This growth accompanies the emergence of real-time interactive and data-intensive applications. Examples include web streaming and cloud gaming, both services where users expect sub-second latency and high-resolution multimedia, highlighting the need for fast and capable data processing.

Cloud computing has been the backbone that enables these applications to operate at scale. It offers automatic scaling, both horizontally and vertically, a pay-per-use model and, among other aspects, a distributed offering and availability that have established it as the foundational layer supporting modern applications and services [4].

Yet, in parallel, a growing share of internet traffic now originates from mobile devices, which have become the main gateway to internet access [5]. Much of this access is therefore generated and consumed at the network edge. Edge computing brings computational and storage resources closer to the end users, allowing for much lower latency in delivering services and applications [6, 7]. Since data can be processed and stored locally, only a subset of it is sent to the cloud [6].

Despite these advantages, edge resources are dispersed, which translates into less capacity per location and greater complexity in managing the distributed edge servers. As Satyanarayanan puts it, "the dispersion inherent in edge computing raises the complexity of management considerably" [7, p. 37]. This complexity is aggravated by considerable variation in user demand over time and by the heterogeneous resource requirements of the requests served. Luo et al. further highlight that, without efficient resource scheduling, this distributed nature leads to resource waste [8]. Resource management therefore becomes increasingly important, since the limited capacity of each location must be adjusted over time to suit the constant fluctuation of user demand.

Scale-up alone does not guarantee usable capacity instantly. The orchestration must detect the demand shift, deliver that evidence to the decision point, provision the right kind of capacity, and admit traffic to it, and every step hands over to the next across a boundary between functions. This sequence is the demand-to-capacity chain, the path that converts a change in overall user demand into capacity that serves that demand: monitoring and telemetry observe the shift, a decision point selects a scaling action, the action provisions new capacity, and that capacity reaches a readiness state (it can answer requests, not merely run) and starts receiving traffic, which is the point at which it becomes usable. The three interfaces between its steps, highlighted in Figure 1.2, are the subject of this thesis.

An *interface* is a boundary between two steps of the chain at which two functions meet, and what crosses it is the evidence, the command, or the state the next step acts on. A *handover delay* is the latency between a function's handover and the next step acting on it, and the *coordination gap* is their accumulated effect along the chain. The reviewed literature addresses these boundaries mainly between functions that run under independent control loops, each observing, deciding, and acting on its own.

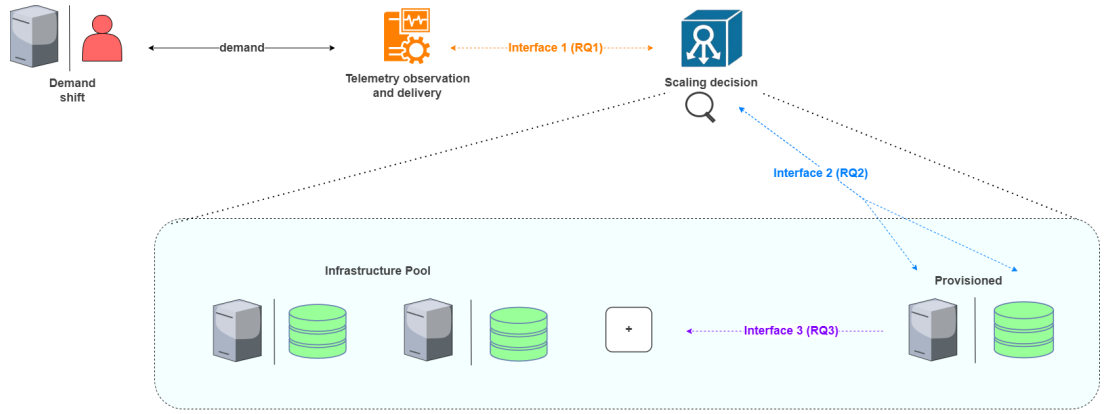


FIGURE 1.2. The demand-to-capacity chain: from a demand shift to usable capacity, with the three interfaces studied in this dissertation highlighted.

The steps described above must be coordinated under the time pressure of an ongoing demand shift. They may run as separate components, each with its own control loop, or within one control process, and the interfaces between them remain in both arrangements. In the reviewed literature the cost of these crossings is acknowledged, but no study varies one crossing on its own to see what difference it makes [9, 10]. The crossings also differ in more than latency. Telemetry can arrive complete and fresh, complete but delayed, or with summaries missing [11]; a scaling action can serve the tier that binds the demand or one that does not [12]; and the state that admits a backend to traffic can be late or inaccurate [13].

This thesis varies the three interfaces of the chain one at a time: how telemetry is consumed by the controller (the telemetry mode); whether the scaling action serves the tier the demand presses on (bottleneck-aware scaling action); and whether a ready backend is admitted on the backend's notification alone, on the notification with a fallback check, through periodic polling, or on the notification with a confirmation check (readiness admission). The platform is organised as two peer Software-Defined Networking (SDN) controllers, one per network domain, and the consuming functions of the chain (telemetry consumption, scaling decisions, and traffic routing) run in the same controller process, so that each interface can be configured and varied on its own rather than observed as an incidental coupling. Telemetry production and aggregation remain external to the controller, so how aggregated telemetry is consumed is an explicit boundary, while scaling and admission are exposed as lifecycle hooks.

The thesis claims that the property of each interface decides whether and how new capacity reaches users during a demand shift, and that each property can be measured on its own. It does not claim that the architecture is superior to existing orchestration platforms such as Kubernetes or NFV Management and Orchestration (MANO) platforms, and it claims no aggregate cost along the chain.

1.2. Objectives

The main objective of this thesis is to contribute to more efficient resource orchestration for edge services by characterising the interfaces of the demand-to-capacity chain under controlled demand regimes, and by establishing, for each interface, which property of the crossing decides

the outcome and under which conditions: how telemetry is consumed by the controller, whether the scaling action serves the tier the demand presses on, and whether a ready backend is admitted on its notification alone, on the notification with a fallback check, through periodic polling, or on the notification with a confirmation check. The platform keeps controller-side coordination in-process and retains the consumption of telemetry as an explicit, configurable boundary, with scaling and admission exposed as lifecycle hooks. Sharing one process couples the components, so making each interface independently configurable is itself a deliberate design choice, and it is what allows the telemetry mode, the scaling action, or the admission of ready capacity to be varied and measured.

1.3. Research Questions

This thesis seeks to answer the following three research questions:

- *RQ1*: How do the four telemetry modes (push, delayed push, polling, and sampled push) affect how the controller scales the service when load rises?
- *RQ2*: How do the four scaling policies (no scaling, compute-only, storage-only, and bottleneck-aware) affect service recovery and resource use under a compute-bound press, a mixed press, and a data-access press?
- *RQ3*: When a new backend is provisioned, how do the four admission modes (backend notification, notification with a fallback check, periodic polling, and notification with a confirmation check) affect whether and when the new capacity starts serving requests and what users experience, under three message conditions: the message arrives intact, never arrives, or arrives before the backend can serve?

1.4. Research Methodology

The work presented in this dissertation follows the Design Science Research (DSR) methodology [14], [15] proposed in [16], and demonstrated in Figure 1.3.

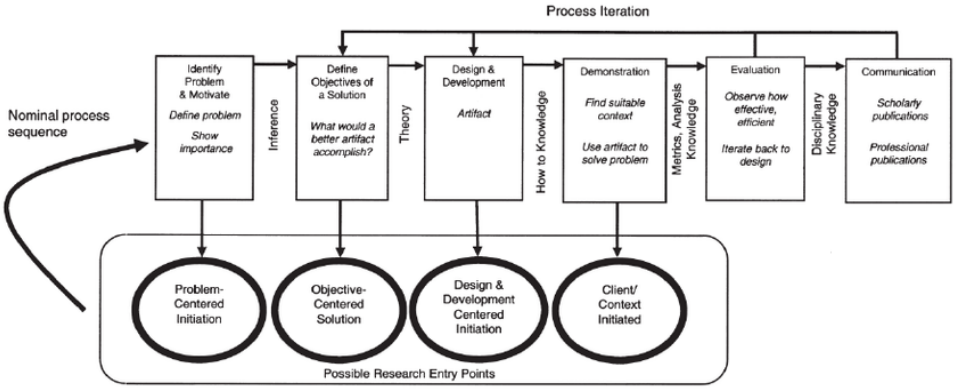


FIGURE 1.3. Design Science Research Methodology

Design science produces and evaluates purposeful artefacts that address certain real-world problems. The research contribution corresponds to the knowledge gained by designing the artefact and evaluating it against defined objectives. In Design Science Research Methodology (DSRM) the research is organised into six categories: problem identification and motivation,

definition of the objectives of a solution, design and development, demonstration, evaluation, and communication.

This dissertation follows a problem-centred approach, in which the problem motivates the research and is defined as the coordination between the functions that orchestrate an edge service, where handovers that are late, incomplete, mismatched, or unverified can lower orchestration efficiency and the quality of service users experience. The problem definition is grounded in the state of the art reviewed in Chapter 2. The six activities unfolded as follows.

(1) *Problem identification and motivation.* Edge services whose monitoring, scaling and routing modules operate as independent components depend on information that crosses the boundaries between them, and how those crossings are operated can degrade orchestration efficiency. Section 1.1 introduces the problem, and Chapter 2 establishes that these functions are typically studied in isolation, which motivates and delimits it.

(2) *Objectives of a solution.* As defined in Section 1.2, the solution is an experimental platform, not a novel algorithm, whose purpose is to make the interfaces of the demand-to-capacity chain measurable.

(3) *Design and development.* The artefact is the single-process platform described in Chapters 3 and 4. It is built from two peer SDN controllers (OS-Ken/OpenFlow), one per network domain, each hosting telemetry consumption, elasticity decision-making and the double-VIP routing model, with Docker containerised service backends and a MongoDB replica set per domain as the storage tier, scaled through native replica-set membership.

(4) *Demonstration.* The artefact was deployed in an emulated two-domain edge environment and driven with scheduled workload phases reproducing demand shifts, demonstrating the chain end to end from demand observation through scaling action to traffic admission, with the tested interface varied across runs.

(5) *Evaluation.* Each research question was evaluated in a dedicated experimental campaign, under a pre-registered measurement and statistical procedure, reported in Chapter 5. A campaign is the set of runs that answers one research question by varying one interface while the others are held fixed, and each run executes the campaign's workload as an experimental unit. The workloads are issued by a scheduled, open-loop driver that applies the offered request schedule independently of the responses it receives, so the offered load is preserved when the service degrades. The primary comparisons are detailed in Section 5.1, and the threats to validity are discussed in Section 6.3.

(6) *Communication.* The problem, the artefact and the findings are communicated through this dissertation, its public defence, and any resulting publications.

1.5. Main Contributions of this Dissertation

This dissertation makes the following contributions:

(1) An experimental characterisation of four telemetry modes (push, delayed push, polling, and sampled push) and of how they affect how the controller scales the service when load rises (RQ1).
(2) An experimental characterisation of the bottleneck-aware scaling policy, more specifically how deciding between compute and storage scale-up from the observed bottleneck, compared with fixed single-tier policies, affects relief of the pressed tier, service recovery, and resource use under a mixed press and a data-access press (RQ2).
(3) An experimental characterisation of readiness admission: how the four admission modes affect whether and when new capacity starts serving requests, under three message conditions (RQ3).
(4) A comparative synthesis of the three interfaces studied (telemetry modes, scaling action, and readiness admission) under their respective controlled workload regimes, identifying which effects were robust, conditional, or not established.

1.6. Dissertation Structure

This dissertation is organised into six chapters. Chapter 1 introduces the research: context, problem, objectives, research questions, methodology and contributions. Chapter 2 presents the review methodology, the platform technologies, the three interfaces, and the research gaps that motivate this dissertation. Chapter 3 presents the architecture and design, and Chapter 4 its implementation and validation. Chapter 5 reports the three experimental campaigns and the synthesis across them. Chapter 6 restates the findings and contributions, discusses the limitations, and outlines future research.

CHAPTER 2

Background and Related Work

This chapter presents the background and related work that motivated the research, selected through the search strategy described in Section 2.1. The review is separated into two parts. First, the technologies and concepts used to build the experimental platform are explained and surveyed, each introduced for what it is and why it suits the edge scenario. These are, more specifically, edge computing, SDN and OpenFlow, containerisation, and NoSQL document stores with a focus on MongoDB.

On top of these technologies runs resource orchestration, the layer that joins monitoring and telemetry, auto-scaling, and load balancing into one service. The review then focuses on the three interfaces of the demand-to-capacity chain: how a load rise is detected, how the scaling action is chosen when several are available, and how new capacity is admitted to traffic.

For each interface the review asks the same two questions: does the reviewed work vary this interface, and does it measure what happens downstream? The literature acknowledges these interfaces but has not studied them one at a time. Section 2.6 reviews the platforms these functions run on, and Section 2.7 collects the gaps.

2.1. Review Methodology

Searches were executed in Google Scholar and IEEE Xplore for publications from 2016 onward, with the number of citations taken into consideration, and works reached through Google Scholar were retrieved from IEEE Xplore, ACM Digital Library, Springer, and ScienceDirect. Each query combined domain terms (monitoring and telemetry, auto-scaling and data, load balancing, and platform context) with the interface names. A work was retained when it addressed at least one of the three interfaces in software-defined, containerised, or edge settings, when it was a survey or taxonomy framing one of the domains, or when it was a seminal work reached by citation chaining. Duplicates, non-English work, and work whose focus lay outside the orchestration of service capacity were excluded.

Screening and citation chaining produced the reviewed literature, 35 works: 27 address an interface and eight supply platform context, treated as context rather than as independent evidence of an RQ gap. The 27 are organised by the area each informs: telemetry, the scaling decision, readiness admission, and orchestration. Appendix D lists each work, the property it varies, and the question it leaves open, and Section 2.7 records the synthesis.

2.2. Platform Technology Background

Edge computing places computation and storage between end devices and centralised data centres, close to where data is produced. Latency-sensitive work runs at the edge while large-scale analytics and long-term storage stay centralised [6, 7]. Dispersing resources raises management

complexity, because heterogeneity, scarcity, and demand that varies over time can degrade orchestration efficiency, and scheduling that does not follow the workload leads either to over-provisioning or to under-provisioning [8, 17]. The service evaluated here runs in an emulated edge environment, with a capacity that must follow fluctuating demand.

Software-Defined Networking separates the control plane, which decides how traffic is handled, from the data plane, which forwards packets, so that forwarding is programmed centrally from a controller, a decoupling from which edge deployments benefit [18]. The OpenFlow protocol implements it through match-action rules installed on the switches and is the most widely used SDN protocol in the research community [19], whereas a language such as P4 lets the operator define how packets are parsed [20]. Here, that programmability keeps routing decisions inside the platform's control loop: which backend serves a request, when newly scaled capacity joins the traffic path, and which storage member serves a read are decided by the controller and enacted as flow rules. Containers package an application with its dependencies and share the host kernel, which makes them lightweight and quick to start compared with virtual machines. Docker is adopted as the containerisation tool.

Document-oriented NoSQL stores hold schema-flexible documents rather than rows of relational tables, and because related data is embedded in one document and joins are avoided, records can be partitioned and replicated across servers. MongoDB organises redundancy around the replica set. One member is elected primary and serves the clients' writes, recording them in an operation log that the secondaries apply asynchronously, and a failed primary is replaced by election without operator intervention [21]. Members can be added and removed while the set keeps serving, which makes the replica set a controllable unit of storage capacity, and this dissertation scales the storage tier that way.

2.3. Telemetry

Monitoring collects the resource and load metrics of a running service, and telemetry carries them to the controller. Every scaling decision is taken from that telemetry, so when it is late or incomplete the reaction is late or based on an incomplete picture, and in an edge deployment the metrics are produced across several nodes before they reach the decision point. How telemetry arrives is therefore the first interface studied in this thesis.

Telemetry is delivered in one of two common patterns, and both carry the same object, a telemetry summary that aggregates the reports of the nodes over a fixed reporting period. In the event-driven pattern the producer sends each summary as it closes, and in polling the consumer reads the latest summary at a fixed interval. Either pattern can reach the consumer late, or with summaries missing, according to how it is configured. A read at the same interval as the summaries delivers every summary, at most one period old, while a slower read, or a producer that samples or holds summaries back, loses or delays them. The four telemetry modes compared in Chapter 5 are these two defects separately and together: push (every summary, on time), delayed push (every summary, 30 s late), sampled push (one summary in three, on time), and polling (only the latest summary at each 30 s read, so it is late and misses two in three).

Monitoring is conventionally pull-based, with the consumer requesting the latest sample on a fixed interval [11], as in OSM-based NFV, where the collector polls the infrastructure into Prometheus and the control modules read it [10]. AdapPF comes closest to the delivery question, varying the Prometheus scrape interval in a Kubernetes federation. A coarse sixty-second interval degrades scheduling accuracy relative to a five-second one, and an adaptive interval restores it while reducing monitoring traffic [22]. Yaseen records the "visibility gaps" that polling leaves between reads and calls for monitoring tightly integrated with control [11]. Hung et al. show that collection and routing can run at one controller, but their loop does not scale the service [23]; Usman et al. leave the choice of measurement interval open [24]; and Zhou and Yong change what is monitored, from CPU load to application errors, without varying the delivery [25].

Within the reviewed literature, no work separates late telemetry from missing telemetry, and none follows the effect of a telemetry mode through the service to what its users experience. AdapPF varies freshness and measures a downstream consequence, but does not separate late from missing updates. Separating delay from loss, and measuring both through the service, is the gap this dissertation fills.

2.4. Auto Scaling

Auto-scaling adjusts the resources allocated to a service to the load it receives with capacity being provisioned as demand grows and released as it recedes, so the service keeps serving without holding resources it does not use. Strategies are grouped into horizontal scaling, which changes the number of units, and vertical scaling, which changes the capacity of a single unit, with hybrid strategies combining the two [26]. Horizontal scaling suits the edge scenario, because a site's capacity is bounded by the nodes deployed there, so enlarging an individual unit has a hard ceiling. This dissertation follows that direction, where adding a unit of capacity to a tier is scale-up and removing one is scale-down.

Qu et al. classify auto-scalers by scaling indicator, resource estimation method, and timing of the action, and Taherizadeh and Stankovski present a taxonomy for edge computing whose dimensions include adaptation intervals and architectural support for multi-tier applications, noting that a scaling approach should account for different provisioning policies at each tier. Both take the scaling of service instances, whether virtual machines or containers, as the unit of analysis. The questions the field asks are when to scale and how many instances to add, and which kind of capacity is provisioned is not among the classification dimensions [9, 27].

With SDN the traffic side is added, but the scaled resource remains the one the service function is built from. Llorens-Carrodeguas et al. couple SDN-based traffic steering with horizontal scaling of virtualised network functions, so that the controllers re-program the forwarding paths while instances of the same pre-dimensioned function pool are activated or deactivated [10]. The decision reduces to whether and how many instances of that one function to activate. In such a service, however, the storage tier is itself a live capacity. When the pressure sits in the data rather than in the function instances, adding instances does not relieve the storage tier.

Predictive and learning-based approaches sharpen the same decision without widening it. Toka et al. formalise the Kubernetes horizontal autoscaler's control model for edge clusters [28], and Ju et al. and Cheng et al. forecast workload to adjust replica counts ahead of demand [29, 30], where the optimisation lies on the forecast and on the number of compute replicas.

Scaling one resource does not always make the service better, because the pressure may sit elsewhere. Nikravesh et al. show in a three-tier web service that the scaling decisions taken for the business tier are insensitive to database capacity, yet reducing that capacity raises the rate of SLA violations, so service quality can be governed by the tier the scaling decisions ignore [12]. Dogani et al. make the same observation in their survey, adding that resources do not necessarily raise throughput when the bottleneck lies elsewhere [31], and Waterfall accounts for how scaling one microservice can shift the bottleneck downstream. PBScaler identifies the microservice whose degradation propagates through the invocation graph and optimises replica management around it [32], but the bottleneck it locates is always a microservice and the action is always replica scaling.

The closest reviewed precedent to choosing between resource tiers is the provisioning model of Li et al., which adds both cloud compute instances and data replicas in cloud-assisted edge systems from a forecast of future workload [33]. Both actions are available, but the trigger is a forecast rather than a live observed bottleneck, and the outcome is measured in tenancy cost.

Overall, the kind of capacity added matters. Compute replicas do not relieve a storage bottleneck, and database capacity can govern SLA violations while compute-tier scaling remains unchanged. In the reviewed studies the kind of capacity an action adds is an architectural given, settled by forecast where a second kind exists, and runtime bottleneck identification still acts on compute replicas only. This dissertation isolates that choice and measures its effect on relief of the pressed tier, service recovery, and resource use.

2.5. Readiness Admission in SDN Load Balancing

Load balancing distributes traffic among a set of available backends. In SDN networks the controller installs the forwarding rules that steer each flow to a selected backend, and a policy decides which backend receives it.

Surveys group the techniques into conventional and AI-based families [34] and into static and dynamic mechanisms, noting that controller state can fall out of step between synchronisations [35]. Abdelltif et al. place the balancer on the controller itself, holding per-server load current over the OpenFlow pool [36]. Taken together, these studies treat load balancing as a selection problem, choosing among backends by load, path properties, and resource utilisation, with the set receiving traffic taken as given.

The membership of that set, however, is not fixed. Backends are added as demand rises and removed as it recedes, and capacity becomes usable only when traffic can reach it. A scaling action provisions a backend, but the provisioned backend serves nothing until it is admitted to the set. The scaling loop that began with the demand shift closes at that admission, and the

time a ready backend spends outside the set is therefore an admission delay. The reviewed work recognises the time this takes, but does not isolate the admission step.

Kurma, a load balancer for geo-distributed storage, works at a granularity of seconds so that it can "work in tandem with modern elastic controllers, thereby reducing over-provisioning and SLO violations incurred during provisioning delays" [1, p. 386]. The delay between provisioning and usable capacity is therefore a recognised cost, yet the time from ready to serving is not measured on its own there.

Llorens-Carrodeguas et al. couple auto-scaling with load balancing in an SDN redirector for a transparent VNF cluster, resizing the pool while traffic is served and removing unhealthy instances from status obtained over the MANO monitoring path [10]. Dynamic membership is therefore already a legitimate concern, but the evaluation reports activation for the whole system, so the time from ready to serving is not isolated.

Managing the pool itself is an established problem. Dreibholz and Mazumdar apply the IETF Reliable Server Pooling framework to a pool spanning edge and multi-cloud resources, where a member registers on join and de-registers on leave while a registrar keeps its availability current through periodic keep-alive checks; their analysis notes that "propagating load updates takes time, leading to temporary inaccuracy" and leaves the mitigation to policies that discount stale pool state [13, p. 6].

The set of backends receiving traffic and their true serving state can fall out of step: a late or lost change leaves ready capacity outside the set, and a backend admitted before it can serve receives requests that fail.

This dissertation treats admission into that set as the variable, holding selection among admitted backends fixed. The four admission modes require different evidence before a backend is admitted (backend notification, notification with a fallback check, periodic polling, and notification with a confirmation check). It measures how they affect whether and when the new capacity starts serving requests and what users experience, under three message conditions: intact, lost, or premature. Chapter 5 evaluates the four modes under one regime.

2.6. SDN-Based Orchestration Platforms

Resource orchestration coordinates the compute, network, and storage resources a service uses, and at the edge it is more demanding because each site holds scarce capacity and the demand on it fluctuates. Earlier work documents SDN at the edge as architecture or proof of concept rather than as coordination inside a running system [18, 37, 38], and resource optimisation in SDN-based edge environments remains among the field's continuing challenges [39].

SDN-based orchestration is an established arrangement for this layer, both at a single control point, as in the SDN redirector of Llorens-Carrodeguas et al. [10] and the telemetry-and-routing loop of Hung et al. [23], and across sites, as in OneEdge [40], McK8s [41] and Kurma [1]. The coordination is built into each system, but the delay it carries is not isolated. None of these evaluations attributes the time taken along those paths to the individual handovers between observing demand, deciding, provisioning, and admitting capacity. What the reviewed

systems do not do is hold the whole control process in one executable and expose those handovers as configurable interfaces, which is the arrangement Chapter 3 presents.

2.7. Synthesis and Research Gaps

Across the reviewed literature, the cost of these crossings is acknowledged, but never measured on its own. Where a cost is measured, it is measured for a whole system. Kurma reports provisioning delay as one combined quantity [1], OneEdge breaks end-to-end deployment latency into stages without attributing it to a single handover [40], and Llorens-Carrodeguas et al. report only whole-system activation [10]. AdapPF comes closest, varying monitoring freshness and measuring a downstream consequence, but without separating late from missing updates [22], while the placement literature names the interval between arrival and readiness as launch time and leaves readiness-to-traffic outside its scope [42].

No reviewed work varies the other properties of these crossings either. Table 2.1 condenses these findings into the three interfaces and the question each leaves open.

TABLE 2.1. The three interfaces of the demand-to-capacity chain and the questions they leave open.

Interface	Open question
Telemetry to decision (RQ1)	Does the controller scale differently when updates arrive late but complete, versus when updates are missing?
Decision to scaling action (RQ2)	When compute and storage actions are both available, how much does choosing the action matched to an observed compute-bound, mixed, or data-access-bound bottleneck change relief of the pressed tier, service recovery, and resource use relative to fixed single-tier policies, and where do the effects concentrate?
Ready backend to traffic (RQ3)	When a newly provisioned backend becomes ready, how do the four admission modes affect whether and when the new capacity starts serving requests and what users experience, under the three message conditions?

The research questions of Section 1.3 are the experimental form of Table 2.1, and Chapter 3 presents the platform that makes them measurable.

CHAPTER 3

Architecture and Design

This chapter presents the architecture and design of the proposed orchestration platform for edge services. The design provides a platform that runs controller-side telemetry consumption, elasticity coordination, and traffic steering in one controller process per domain. The telemetry mode, the scaling decision, and the transitions from scaling action to traffic admission remain explicit observation points.

Section 3.1 defines the requirements that guided the proposed solution, and Section 3.2 presents the layered organisation of the platform and its two-domain deployment. Section 3.3 describes the distributed elastic allocation of compute and storage resources, Section 3.4 the monitoring and decision engine, and Section 3.5 the control workflow. Together, these sections realise the design requirements: the domain organisation and the data paths (DR1, DR4, DR5), the elastic allocation and the capacity changes (DR3, DR7), the observation and decision path (DR6, DR10), and the execution contexts and instrumented chain of the control workflow (DR2, DR8, DR9).

3.1. Design Requirements

The platform was designed to support edge services whose demand and resource requirements vary over time. The requirements below follow from the coordination problem and the research gaps established in Chapter 2, and guide the design presented in the remainder of the chapter.

DR1: Domain-scoped control. Each network domain must be operated by its own controller, which observes that domain's demand and acts on that domain's resources. Peer controllers exchange resource state, but the operation of a domain must not depend on a central entity.

DR2: Single-process control loop. Telemetry consumption, the scaling decision, load balancing, and the capacity changes must run in the same controller process, sharing their state directly rather than through inter-service handovers.

DR3: Dual-tier, demand-driven elasticity. Compute and storage capacity must each follow their own demand and scale independently, and storage capacity must be changeable at run time from the platform's own code while the service keeps serving.

DR4: Stable addresses in front of the tiers' service interfaces. Clients and servers must address stable virtual addresses, one for the service and one for storage reads, and the controller must enforce each routing decision on the network path, so that capacity changes never change the point of access. Each address must front what its tier serves rather than a node. A service request must be a call to the API served by the compute replicas, answerable

by any admitted replica, and a storage read a request through the interface the storage tier exposes, answerable by any admitted member.

DR5: Two-level load distribution. Request load must be distributed across the compute replicas and storage reads across the storage members, using the candidates' reported state, with only admitted capacity eligible.

DR6: Hierarchical reporting with freshness bounds. Edge servers and storage members must report to their domain's telemetry aggregator, and the controller must consume the summaries formed from these reports. Each summary must carry when each node last reported, and a node silent beyond a bound must be treated as absent, so that the controller acts only on capacity it can observe.

DR7: Staged capacity changes. Capacity must enter and leave service through staged transitions. Added capacity must be provisioned and registered before it can receive traffic, and removed capacity must be withdrawn from routing and drained of in-flight requests before it is released. The routing pools must contain only capacity that can serve.

DR8: Non-blocking steering. Packet handling must never wait on slow infrastructure work such as provisioning or removal.

DR9: Instrumented transitions. Every capacity change must be a sequence of explicit, timestamped transitions (provisioned, ready, admitted, serving), and every served request must be attributed to the node that served it, so that the chain from observed demand to the first request served by new capacity can be reconstructed.

DR10: Configurable interfaces. The telemetry mode, the scaling action, and the admission mode must be independently selectable configurations of the platform.

3.2. Architecture

Within each domain the platform is organised in three layers (Figure 3.1). The control layer holds the controller, which consumes the telemetry summaries, performs load balancing, and decides the scaling actions. The data plane layer holds the software switch, which carries the client requests and the storage traffic along the path the controller decides. The resource layer holds the edge servers that serve requests, the storage members that hold the service state, and the telemetry aggregator that forms the summaries for the controller.

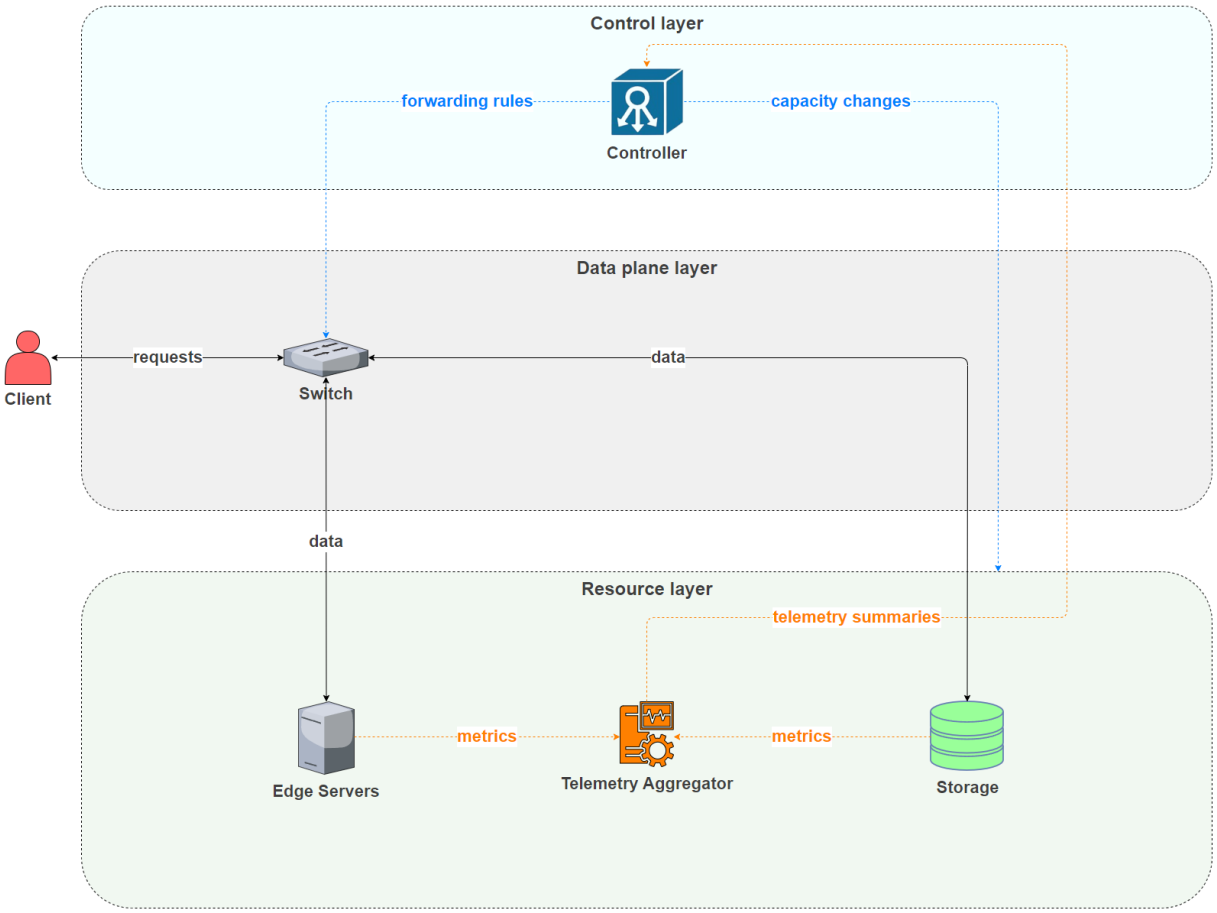


FIGURE 3.1. Layered organisation of the platform in one network domain: the controller of the control layer consumes the telemetry summaries, decides how the traffic is routed, and issues the capacity changes to the resources. The data plane layer carries the client requests and the storage traffic, and the resource layer holds the edge servers, the storage members, and the telemetry aggregator.

3.2.1. Two-Domain Organisation

The platform is organised as two network domains joined by an inter-domain data path, representing the two domains of the deployment. Each domain contains client workloads, a software data plane, a pool of edge servers, a replicated storage tier, a telemetry aggregator, and a controller. The peer controllers exchange topology information, and each consumes the telemetry summaries produced by the aggregators of both domains. Each controller runs telemetry consumption, elasticity coordination, and traffic steering in the same process for its own domain. Where this thesis says node without qualification, it means one deployed unit of either tier: an edge server of the compute pool, or a storage member of the replica set.

Both domains run on one host in the evaluated deployment, so the control plane is local and only the inter-domain data path crosses domains. The evaluated demand stays within a domain, so the two-domain organisation is exercised as a cross-domain-aware control plane with a single-site data path.

3.2.2. Virtual IP Data Paths

Clients access edge services through a per-domain Virtual IP (VIP). Service requests are addressed to the virtual service address `VIP_SERVER`, and edge servers route storage reads through the virtual data address `VIP_DATA`. This separation represents service routing and data access as distinct routing surfaces.

Neither address names a node. A request to `VIP_SERVER` is a call to the service API, answerable by any admitted compute replica, and a read to `VIP_DATA` is issued through the storage tier's data interface and answered by whichever member the controller selects. The client and the edge server address the tier, and the controller decides the node.

The controller is responsible for both routing surfaces. For each new connection it selects the unit that will serve it among the admitted units of the relevant tier, and it places that choice on the network path. Established connections keep their unit, so a capacity change does not reroute traffic that is already being served. Load balancing may select a backend in either domain (a platform capability held constant in the evaluation), while cross-domain storage placement is not exercised.

Load balancing is cost-based, for both service backends and storage members. The controller compares the candidates' reported resource state under configurable weights and selects the lowest-cost unit, preferring local candidates at comparable state. For storage reads observed on the virtual data path, the same mechanism selects a replica-set member. The metrics of the comparison, and how the selection is enforced on the path, are described in Chapter 4.

A request's latency decomposes into the *pre-dispatch wait*, from the client issuing the request to its dispatch in the serving process, and the *edge service time*, from dispatch to response finalisation, which decomposes into a *data-access* and an *edge-processing* component. Their sum is the request's *user-visible latency*, the client-side round trip. The two edge-service components are the tier-specific latency signals the scaling policy observes (Section 3.3.1).

3.2.3. Control and Telemetry Paths

Edge servers and storage components emit resource and request observations to their local telemetry aggregator, and each aggregator forms summaries for the controller. These summaries inform both load balancing and elasticity decisions, while topology exchange maintains awareness of peer-domain resources. The aggregation-to-controller link is the configurable telemetry interface.

The architecture therefore separates the request data path, the telemetry observation path, and the slow infrastructure-management path while retaining their coordination within each domain's controller process. The implementation of these paths is described in Chapter 4.

3.3. Distributed Elastic Allocation of Resources

The platform scales capacity along two dimensions: compute replicas (also referred to as backends in the routing context), which serve requests, and storage members, which hold the service state. Each dimension has its own observation signals, its own action, and its own admission and

release procedure, so that pressure on the two tiers is addressed separately. The two dimensions are summarised in Figure 3.2 and described in turn.

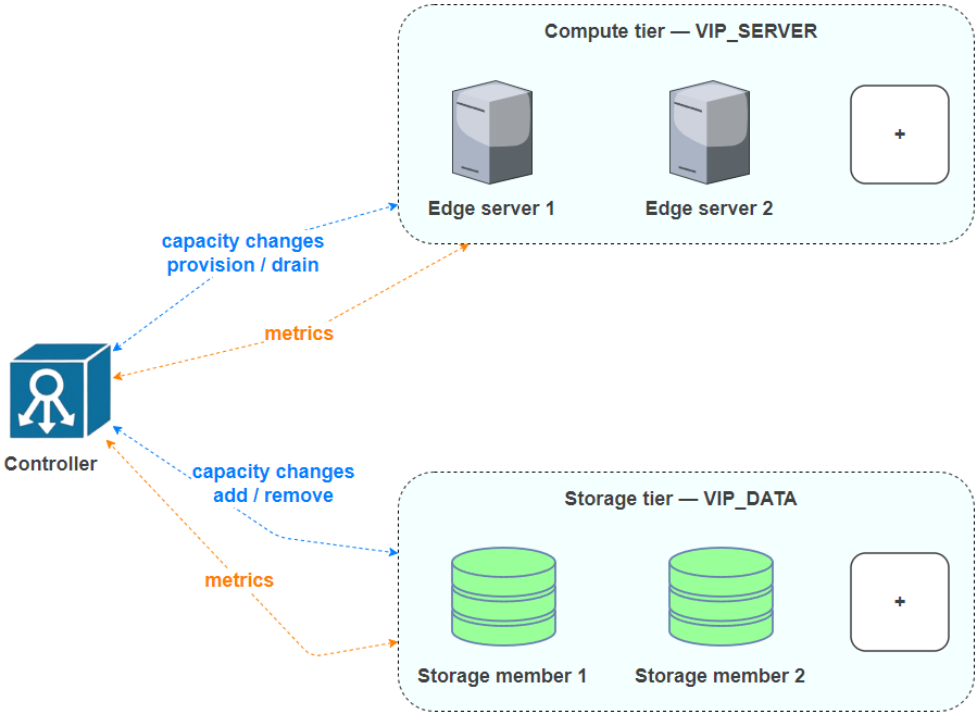


FIGURE 3.2. Elastic allocation of resources: each tier is scaled on its own observation signals, with compute replicas provisioned and drained and storage members added and removed through replica-set membership, both actions issued by the controller over the infrastructure-management path.

3.3.1. Scaling Policy

The policy is applied per resource tier, with tier-specific signals and parameters. Each tier is observed through two signals: a utilisation signal, which in this platform is the CPU utilisation of the tier's units, and a latency signal. For each summary w, let $c(w)$ denote the summary's utilisation and $t(w)$ its typical latency (the tier's latency statistic over the summary, configurable as the mean or the median). Each signal is first mapped onto the unit interval through two calibration parameters, a floor and a span. A value at or below its floor contributes nothing, a value at or above the floor plus its span contributes the maximum, and intermediate values contribute proportionally. The floor separates normal operation from pressure, and the span sets how far the signal must rise before its contribution saturates. Because both signals are mapped onto this bounded scale, they contribute on common terms despite different units, and a single extreme summary cannot dominate the score. The two normalised components are combined

with weights w_{util} and w_{lat} into the degradation score

$$\begin{aligned} s(w) &= w_{\text{util}}\,\hat{c}(w) + w_{\text{lat}}\,\hat{t}(w), \\ \hat{c}(w) &= \min\left(1,\ \max\left(0,\ \frac{c(w)-c_0}{c_\Delta}\right)\right), \\ \hat{t}(w) &= \min\left(1,\ \max\left(0,\ \frac{t(w)-t_0}{t_\Delta}\right)\right), \end{aligned} \tag{3.1}$$

with the per-tier weights configured to sum to one, so the score lies between zero and one. A signal at or below its floor contributes nothing and is treated as normal for the tier. c_0 and t_0 are those floors, and c_Δ and t_Δ the rises above them at which each contribution saturates. The normalised components $\hat{c}(w)$ and $\hat{t}(w)$ therefore lie in $[0,1]$. The floors, spans, weights, and threshold parameters are configured independently for each tier, so the policy can be calibrated to each tier without changing the structure of the score. For the compute tier the signals are CPU utilisation and request-processing latency, while for the storage tier they are storage-member CPU utilisation and data-access latency. These signals feed only the scaling decisions, while load balancing uses the reported resource state described in Section 3.2.2.

A summary counts as degraded when its score reaches an adaptive threshold, and a scale-up requires sustained degradation across recent summaries. Let $\mathscr{W}_n$ be the set of the last W summaries in which the tier was eligible to act, and R the required number of degraded summaries. The scale-up condition is

$$\begin{aligned} \left|\left\{\, w \in \mathscr{W}_n : s(w) \geq \theta(k) \,\right\}\right| &\geq R, \\ \theta(k) &= \min\left(\theta_{\text{base}} + k\,\Delta\theta + \beta_{\text{peer}},\ \theta_{\max}\right), \end{aligned} \tag{3.2}$$

where k is the number of dynamic units currently in the tier's pool, θ_{base} the base threshold, $\Delta\theta$ the increment per additional unit, $\theta_{\max}$ its upper bound, and β_{peer} the peer relief added to the compute-tier threshold while the peer domain is healthy enough to absorb spillover traffic (zero otherwise). The increment is applied linearly for the compute tier. For the storage tier it diminishes with each additional unit, down to a configured minimum. The threshold follows the number of dynamic units in the tier's pool. Each additional unit therefore faces a higher bar, and the threshold falls back as units are removed. A scale-up cooldown after each committed scale-up and a scale-down cooldown keyed on the most recent scale-up keep the pace of actions bounded, together with a per-tier cap on the number of units. The set advances only when the tier was eligible to act, and it is cleared when a scale-up occurs. A committed scale-up also clears the tier's scale-down window, so a recent scale-up is not immediately reversed.

A summary counts as idle when both signals fall below their idle thresholds,

$$c(w) < \tau_c \quad \text{and} \quad t(w) < \tau_t, \tag{3.3}$$

where τ_c and τ_t are the idle thresholds of the utilisation and latency signals. A scale-down is requested only when idleness is sustained across $R_\downarrow$ of the last $W_\downarrow$ summaries, where $R_\downarrow$ is the required number of idle summaries and $W_\downarrow$ the length of the scale-down window. This window is longer than the scale-up one, which prevents a transient data-bound stall from being misread

as idle capacity. Summaries whose latency exceeds a ceiling are treated as indeterminate and do not count toward idleness.

The policy evaluates each tier per summary, in a fixed order:

(1) the summary is skipped when the tier is ineligible. An infrastructure operation is in progress, the tier is at its cap, or a scale-up cooldown is active;
(2) the degradation score $s(w)$ is computed from the two normalised signals (3.1);
(3) if at least R of the last W eligible summaries reach the adaptive threshold $\theta(k)$ (3.2), the tier requests a scale-up and the summary history is cleared;
(4) otherwise, if both signals fall below their idle thresholds (3.3), idleness is tracked, and the tier requests a scale-down once idleness spans $R_{\downarrow}$ of the last $W_{\downarrow}$ summaries.

The values these parameters take in the evaluation are listed in Appendix A.

3.3.2. Compute

Compute capacity is a per-domain pool of edge servers, each capable of serving any request addressed to the domain's virtual service address. Storage reads performed by a compute replica are routed through the domain's virtual data address, or served from a Tier 1 copy where one is held (the data-placement tiers are defined in Section 3.3.3). Reads are left to the routing path. Compute scale-up applies the policy defined in Section 3.3.1.

A newly provisioned compute replica is recorded through an explicit registration step and becomes eligible for load balancing only once it is visible in the routing pool. How readiness reaches that pool is the admission interface, and it has four modes: backend notification admits on the message the backend sends, notification with a fallback check adds a check after a configured wait, periodic polling ignores the message and checks on a fixed interval, and notification with a confirmation check lets the message wake a check that must pass. The modes admit on evidence of different strength, so a message that arrives before the replica can serve reaches the routing pool unchecked under the modes that trust it and is observed by the modes that check. They are measured under three message conditions, each held for the whole run: the message arrives intact, never arrives, or arrives before the backend can serve. Because load balancing steers new connections while established ones remain with their unit (Section 3.5), admission also marks the point at which the new backend can begin to share the load.

Compute scale-down removes the least disruptive eligible replica, applying the idle condition defined in Section 3.3.1. Only replicas provisioned by the platform are eligible for removal, so the original nodes are left undisturbed. A candidate is considered only while its reports are recent, and among the eligible replicas the one carrying the least request load is removed first. Removal proceeds in two phases: the replica is first withdrawn from the routing pool and signalled to finish its in-flight requests, and only after it drains is it released. A pending drain can be cancelled if load rebounds before the replica exits, favouring fast recovery over aggressive reclamation.

3.3.3. Storage

The storage tier of each domain is a replica set, and storage capacity is scaled by changing its membership, one member at a time. Any admitted member can answer a read, so read load is distributed across the membership. Writes are not distributed. The store keeps a single primary for writes, and the remaining members replicate the data and serve eventually consistent reads. This write placement follows from the store's replication model rather than from a platform design decision. The evaluation adopts it as the tier's consistency contract and does not rely on read-after-write semantics. Storage scale-up is a membership change. The controller provisions a new storage member, which joins the set, and admits it to the data path only once the member can serve reads. Preparation is decoupled from the reaction as a readied member is held in reserve outside the routing pool, and the storage scale-up is the activation of this prepared reserve, which removes the initial readiness wait from the reaction path. The activation is a storage admission, and a new reserve is prepared immediately afterwards.

Admission alone does not put the member to work, because a connection chooses its member when it is opened, so the platform renews the edge servers' storage connections on admission and spreads new connections across members whose reported cost is close to the best.

Storage scale-down removes an idle member while the remaining members keep serving. Only members added by the platform are eligible for removal, so the set's original membership is left undisturbed, and the most recently added eligible member is removed first. The decision applies the idle condition defined in Section 3.3.1 to the storage tier's idle signals (its data-access latency and, in the configurations that enable it, its CPU utilisation).

Beyond the membership change, the platform defines three data-placement tiers: Tier 0 provides direct remote reads over the virtual data path, meaning the replica-set member is deployed outside its primary network domain, Tier 1 holds a partial copy of the data in the consumer domain, and Tier 2 is the replica-set extension described above. Tier 0 and Tier 1 are platform capabilities rather than evaluated treatments. The evaluated storage action is limited to activating the prepared reserve or removing a member of the domain's own replica set.

3.4. Telemetry Delivery and Scaling

The producers and the aggregator that collects their reports are described in Section 3.2.3. Each edge server reports after every request it answers, carrying its own resource state and the experience of that request, and each storage member reports as it serves storage requests, so a busy node reports often and a quiet one is kept visible by its periodic heartbeat. The aggregator produces a summary of the most recent period at a fixed interval. Every summary, including those covering idle periods, is evaluated and recorded. Each summary also carries the time of each node's last report, so a node that stays silent beyond an absence bound is treated as gone instead of being acted upon.

The controller consumes these summaries through an explicit, configurable interface. Its four modes are the two defects of Section 2.3 separately and together, implemented as four configurations of one interface rather than as two transports opposed to each other, and they supply the same summaries in different forms.

Because the decision engine evaluates delivered summaries only, the mode sets the earliest moment a scaling reaction can begin, and because the condition counts the last summaries the engine has seen, missing summaries slow how often it is met as well as when it first can be. The aggregation, the workload, and the decision logic remain unchanged across modes, so any difference in how the controller scales is attributed to the evidence it received rather than to the mechanism that carried it.

Inside the controller, the summaries feed two consumers. Load balancing keeps its candidate state from them, so a unit is selected on its current load rather than on the state it had when it was admitted. The decision engine applies the scaling policy of Section 3.3.1 to each delivered summary, and reads both domains. The tier condition is assessed on the local domain's summary, while the peer domain's condition adjusts the threshold through the peer-relief term of Equation (3.2), so a scale-up is decided on the state of the system rather than on the local site alone. Once a tier's condition is sustained, the engine emits an action request, following the fixed order of Section 3.3.1: the eligibility check, then the degradation score, then the scale-up condition, and otherwise the idle condition.

The decision engine may also be configured to commit to a single scaling action (compute or storage), which is the scaling interface of the demand-to-capacity chain. In the bottleneck-aware configuration, that choice distinguishes between compute-bound and data-access-bound demand and scales the matching tier when it leads by the configured classification margin, resolving close scores towards storage. A per-tier action budget can bound its actions, and the action taken and the actions rejected are recorded as evidence. The platform's capacity events, such as the completion of a drain or the readiness of a storage member, reach the same decision path, so that admission and release follow the same flow as the actions themselves.

3.5. Control Workflow

The control workflow separates three execution contexts that share one process but do not block one another. The packet context handles routing decisions and does not wait on slow work. The observation context evaluates delivered summaries and emits action requests. The action context performs infrastructure changes such as provisioning, drain, and removal.

The workflow traverses the demand-to-capacity chain introduced in Section 1.1. A change in demand alters the request experience reported by the compute replicas, and the summary that contains the change is produced and delivered. The decision engine evaluates the delivered summary. If the tier condition is sustained, the engine emits a scaling action request, which the action context executes. The provisioned unit reaches its readiness state and is admitted to the corresponding routing pool, which for the storage tier is the activation of the prepared reserve. New connections can then be directed to it, while established connections remain where they were placed. The chain completes when the new unit serves its first successful request. Each step is instrumented, so the timeline from demand observation to usable capacity is recorded end to end, from registration and routing-pool inclusion to the first successful request (Figure 3.3). When readiness admission is enabled, a compute replica that does not reach readiness within the configured bound, or that is never admitted within the abandonment bound, is abandoned

and torn down instead of being admitted. Without readiness admission, compute replicas are registered as soon as they are provisioned and no readiness bound applies, so a replica that never becomes usable is reclaimed only by the ordinary capacity checks.

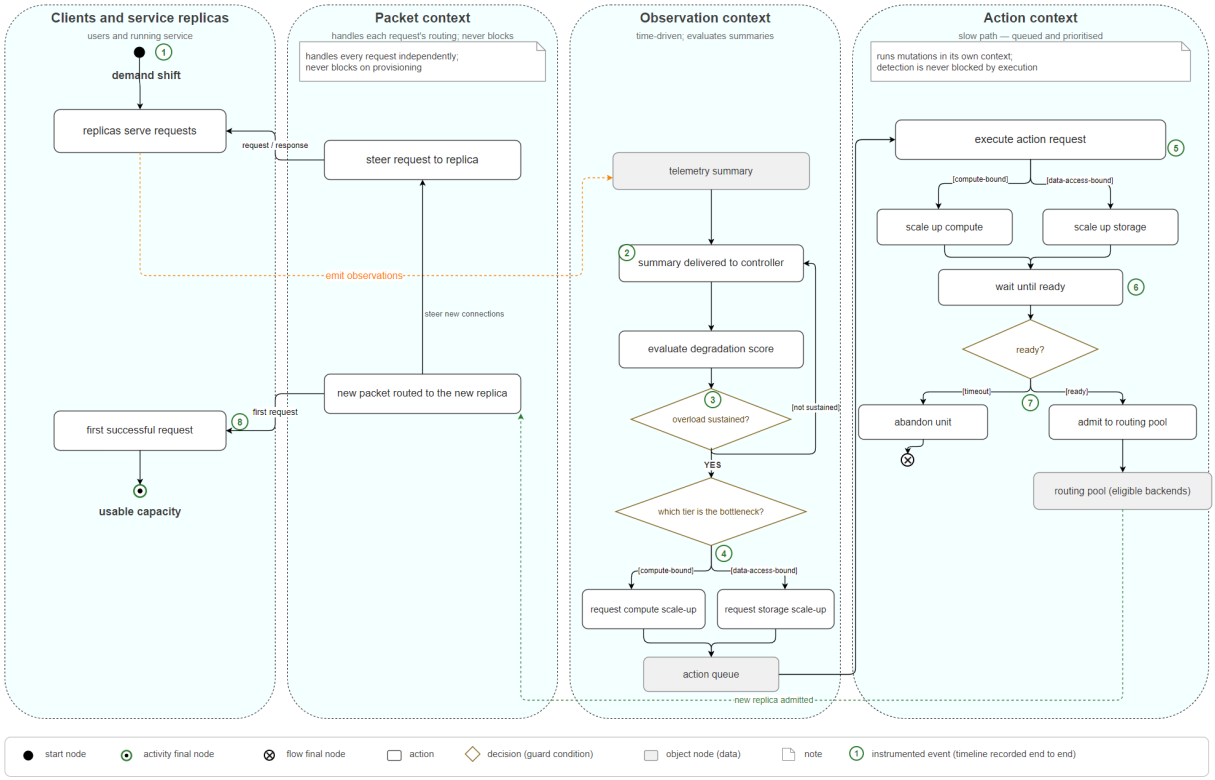


FIGURE 3.3. The control workflow, drawn as a UML diagram. The demand-to-capacity chain runs across three contexts in the same process: the packet context, the observation context, and the action context. The contexts do not block one another and they exchange state through shared structures (the telemetry state, the action queue, the capacity state, and the routing pools). The numbered stages match the chain in Figure 1.2 and show the provisioning direction. Capacity removal follows the same steps in reverse, starting with the drain. The aggregator is omitted between the observations sent by the replicas and the summary delivered to the controller.

State crosses these contexts through shared structures rather than direct calls, and the packet context reads them on each routing decision. Because it never waits on the action context, a long-running change cannot stall request steering, and because the action requests are queued and prioritised, detection is not blocked by execution.

Every request that adds or preserves capacity precedes every request that removes capacity. The cancellation of a pending drain precedes its execution. This execution order is distinct from the scaling decision, which the observation context decides before the request is queued.

Across domains, the peer controllers exchange topology state so that each controller is aware of the peer domain's resources, while acting on its own domain's demand. The concrete realisation of this design, the technologies that implement each context, and the validation of the resulting platform are the subject of Chapter 4. The evaluation of the three interfaces is presented in Chapter 5.

CHAPTER 4

Implementation

Chapter 3 specified what the platform must do. This chapter describes what was built. Section 4.1 presents the testbed and its load source, Section 4.2 the data path on which steering is enforced, Section 4.3 the two scaled resources as implemented, Section 4.4 the observation and decision components, and Section 4.5 the runtime behaviour of the controller. The checks that validate the resulting platform are reported in Appendix B, and the configuration values named in the text are listed per campaign in Appendix A.

4.1. Experimental Infrastructure

The platform runs on a single cloud VM host that holds both domains as Docker containers on emulated networks. The host is a KVM virtual machine running Ubuntu 22.04.5 LTS, with 4 vCPU on an AMD EPYC-Genoa processor at 2.4 GHz, 8 GB of memory, and local SSD storage. Each campaign ran on a single cloud VM host, and the hosts were identical except for a smaller local disk on the telemetry host. The components as deployed are listed in Appendix C.

The two domains are joined by a router container whose inter-domain link carries an emulated, fixed symmetric round-trip time applied with tc-netem (185 ms in the reported campaigns, with the jitter, loss, and rate knobs available and off by default). The emulated link carries the cross-domain data path, and every control-plane link is host-local.

The load is exerted by a scheduled driver that runs on the host beside the testbed. Each client is a Linux network namespace attached to its domain's client network, and the driver issues HTTP calls from it against the domain's virtual service address, so the controller observes the source address and steers each request through the data path. A run instantiates 24 clients per domain, 48 in total, and executes the run's phase schedule (Section 5.1). Active clients issue requests continuously at the phase rate, against users and content items drawn from the seeded snapshot, so traffic generation does not depend on a live database and the run seed fixes the active clients and their request streams.

The delivery of the load is open-loop, so a client keeps issuing its requests on schedule while earlier requests are still outstanding and the offered demand is preserved when the service slows down. A closed-loop driver would instead withhold its next request until the previous one completes, and the two models can support opposite conclusions about a system under load [43]. Three client-side bounds make the failure modes explicit and finite. Each client keeps at most 1024 requests in flight. A request issued while the window is full is recorded as dropped. Every request carries a 300 s client-side cap and is recorded as a timeout at that cap. In-flight requests that outlast the 30 s phase-end drain are abandoned and recorded as cancelled. Each issued

request is written to the run's request record with its phase, kind, targets, issue time, outcome class and latency, which is the raw material of the per-run analyses of Chapter 5.

4.2. Data Path

The data plane runs OpenFlow on Open vSwitch. It must match ordinary addresses and ports, rewrite each connection to its backend, and deliver new connections to the controller, which answers from the same Python process that runs the control logic (DR2). OpenFlow covers this set natively, and it reaches the host's connection tracking, so per-connection identity and address translation live in the kernel rather than in the rules. A programmable data plane such as P4 could express the same steering, but its additional capabilities serve network-level questions that this thesis does not ask, because every decision it evaluates is taken in the controller.

Each domain has two virtual service addresses, one for client requests and one for storage reads. The data plane combines OpenFlow rules with the kernel's connection tracking. On the service path, the controller installs a DNAT/SNAT rule pair for the client and the selected backend. With flow isolation enabled the pair is scoped to a single connection and removed after its response, so every measured request triggers a fresh load-balancing decision. On the data path, a forward rule selects the member and commits a NAT mapping to it, and the kernel records the connection by its five-tuple. Reply packets are handed back to connection tracking, which reverses the mapping, so the identity of each connection lives in the kernel's table rather than in the rule. Two consequences matter for the platform.

Established connections survive rule deletion, which lets the controller bulk-delete a member's forward rules when the member leaves the pool and lets new connections re-select while established ones finish. And because the kernel keeps connections apart by their full five-tuple, the rule can be scoped to the client or, as in the reported configuration, to a single connection of it, so a pooled client spreads its connections across members. Each domain tracks its connections in its own zone, and rules that belong to a domain share a cookie so they can be removed together.

4.3. Distributed Elastic Allocation of Resources

4.3.1. Compute

A compute replica is a Flask application served by the Werkzeug server. It exposes the workload routes, which are the service API that `VIP_SERVER` fronts, a drain endpoint that moves the replica into a draining state, and a readiness endpoint. The readiness predicate is a real service round-trip. The endpoint answers 200 only while a MongoDB ping against the domain's virtual data address succeeds, and the server binds before readiness is declared, so the predicate means servability and the readiness message cannot precede the socket. After each request the replica pushes one telemetry record with the request's outcome and timing (total time, and the MongoDB time split into reads and writes) and its own CPU and memory use, read from the container's cgroup files. With readiness messages enabled it also emits an `app_ready` control event when the predicate first succeeds.

The controller provisions a replica by starting its container with the tier's quotas and the run's environment, attaching it to the domain network through an Open vSwitch port, and recording it as a backend. Removal follows the two phases of Section 3.3. The replica carrying the least request load is removed first, with the lowest utilisation and request-processing latency as tie-breakers, and its network attachment is removed only after its in-flight requests reach zero and a quiet period passes.

Readiness admission is implemented by a component that holds newly spawned replicas in a pending registry and admits them to the routing pool only when its configured rule is satisfied. The rule is selected by a propagation mode, running the four admission modes defined in Section 3.3.2, and the configuration literals behind each mode, with the fallback wait, the polling interval and the confirmation check, are listed in Appendix A. The admission decision is summarised in Algorithm 4.1.

Checks retry at a fixed interval with their own timeout. A replica that is not admitted within the abandonment bound is abandoned and fully torn down. Every admission records its source in the admission log, the message, a poll, a fallback check, or a confirmation check. A check that disagrees with an admission made on the message is recorded as a warning and the admission stands, because the message already admitted the replica. The confirmation check never admits unverified.

Algorithm 4.1 Readiness admission of a spawned compute replica.

enqueue the replica in the pending registry
if mode is `off` **then**
    admit the replica
else if mode is `discovery` **then**
    check `/ready` on each poll and admit on the first success
else ▷ `direct`
    admit the replica on the `app_ready` message
    without a message after the fallback wait: check and admit on success
    with the confirmation check: the message wakes a check, and admission into the routing backend pool requires it to pass
end if
abandon and tear down a replica not admitted within the abandonment bound

Warm-lease priority and slow-start ramps are separate load-balancing behaviours of the routing path (Section 4.5) and are held constant in the readiness experiments.

4.3.2. Storage

The storage tier is built on MongoDB replica sets, because DR3 requires storage capacity to change at run time from the platform's own code while the service keeps serving. MongoDB answers that change as a replica-set reconfiguration command on the connection the platform already holds, with one container per member. The alternatives place the operation elsewhere.

MySQL InnoDB Cluster has it, but only in the AdminAPI of MySQL Shell, a separate management client the platform would have to invoke [44, 45]. A PostgreSQL streaming-replication standby has no membership operation at all, so adding or removing a replica must be built by hand [46].

Each domain's storage is a MongoDB replica set of its own, built in three steps. At network build time, the domain's storage container starts as the set's single member and becomes its primary, and the run's data set is seeded through it. At runtime, a new member starts as a bare `mongod` with a companion sidecar. The sidecar connects to a known member of the set, appends itself as a non-voting member with zero priority so it cannot affect elections, and applies the change with a single `replSetReconfig` call. When the member reaches the secondary state and can serve reads, the sidecar emits the `rs_secondary_ready` control event, and the controller admits the member into the domain's virtual data address, by default on the event, with a telemetry-observed secondary state as the fallback path. While the member serves, the sidecar reports a sample whenever the member's operation counters show activity, carrying replication lag, open connections, and the container's CPU and memory use, and an idle member is kept visible by the periodic heartbeat. Removal reverses the steps. The member leaves the routing pool first, and the controller then performs `rs.remove()` against the primary, waits for the removal to be confirmed, and tears the container down. The interface the virtual data address fronts is the database's own API. The edge server issues its reads through the MongoDB driver against the address, any admitted member can answer them, and the driver never learns the physical membership.

When the prepared reserve is enabled, the activated member is already joined and servable, and its replacement is prepared once the slot is free. The member lifecycle is bounded by the run's caps (Appendix A).

Reads are issued against the domain's virtual data address through per-request leases that the edge server manages per domain epoch. On every storage admission the controller sends a renewal request to each edge server of the domain, and the edge rotates the epoch so that new connections choose a member again. The request is sent in two waves, one immediately and one 45 s later, after the 30 s warm lease, because a connection opened just before the admission keeps its member for the life of that lease. Selection of a member uses a band. Costs within 3 % of the lowest count as equal, the band keeps at least two members, and the choice inside the band rotates. Reads prefer a secondary. Writes are issued directly to the replica-set primary. Selective synchronisation (Tier 1), which holds a partial copy of the data in the consumer domain, and cross-domain storage placement, which places a replica-set member in the peer domain, are capabilities of Section 3.3.3 and are disabled in the reported experiments.

4.4. Telemetry Delivery and Scaling

Each edge server reports after every request it answers, and each storage member reports as it serves storage operations, so the telemetry path is not flooded with idle traffic. The domain's aggregator collects these reports and produces a summary of the preceding 10 s every 10 s. A

heartbeat every 60 s keeps a quiet but alive node visible, and a starting node is visible before its first request.

The configurable interface of Section 3.4 is implemented as alternative sources over the same summaries, so the four modes differ only in how the same summaries reach the controller. Every summary is appended to a durable, sequence-numbered log that the replaying modes read. Appendix C lists the four modes and their implemented sources.

The platform's default source pushes each summary as it is produced, without a delivery guarantee, while the campaign's push mode uses the acknowledged replay, so the campaign can verify that every produced summary arrived. The replaying sources record every summary in a delivery log with its network, sequence number, close time, delivery time, delay, and source mode, which is what the telemetry experiments read. Two kinds of message stay outside the telemetry modes. Control events are the platform's own notifications, forwarded by the aggregator immediately: a replica reporting that it can serve (`app_ready`) or that its in-flight requests have finished (`drain_complete`), a storage member reporting that it is ready to serve reads (`rs_secondary_ready`), and, where per-request flow isolation is enabled, each completed request (`request_complete`), which releases its flow so the next request is routed afresh.

A collector writes per-run resource records for each domain, debug records and per-node files, and samples the CPU and memory of both controller instances. The controller's own records complete the timing evidence: the lifecycle of each provisioned unit, the elasticity events, the admission log, and the decision log, kept per run beside the request record of Section 4.1 and the delivery log above. Inside the controller, the delivered summaries feed the two consumers of Section 3.4, and every decision is written to the decision log for the summary that produced it, together with its evidence. In the bottleneck-aware configuration, the decision engine commits at most one action per summary and records the bottleneck class it assigned, the per-tier evidence, the action taken and the actions rejected, and the budget counters. Absence detection and scale-down evaluation run on a fixed ticker once per delivered summary rather than when it arrives. The configuration surface is environment-driven, and its values are listed in Appendix A.

4.5. Control Workflow

Each domain runs its own controller instance. The three execution contexts of Section 3.5 are implemented as one cooperative event loop and two native threads. The packet path handles flow decisions inside the OpenFlow event loop and never blocks on infrastructure work. The telemetry consumer runs on the same event loop. It receives the summaries, updates the per-backend and per-member state, evaluates the tier conditions, and posts typed alerts to a priority queue. The elasticity thread is a daemon thread that blocks on that queue and performs the infrastructure changes, and the readiness gate runs its own worker thread for checks.

Each alert identifies its resource tier and its action, and the order of Section 3.5 is refined by tier. Among additions, a storage scale-up precedes a compute scale-up, and among removals, a storage scale-down precedes a compute scale-down.

State crosses the contexts through the shared structures of Section 3.5. The telemetry consumer writes the per-node state and the alerts between its yield points, the elasticity thread hands its results to the routing pools through the same registration the packet path reads, and the packet path reads the pools on each flow decision.

When no rule matches a connection's first packet, the packet is sent to the controller, which selects the lowest-cost unit among the eligible candidates under the cost function of Section 3.2.2.

For backends the cost uses CPU utilisation, memory usage, request count and hop distance. For storage members, CPU utilisation, memory usage, connections, replication lag and hop distance. Each metric enters as a fraction of the largest value observed among the eligible candidates, so metrics with different units contribute on a common scale, ties at the lowest cost are broken in round-robin, and a peer-domain candidate is always assigned more hops than a local one, so local candidates are preferred at comparable resource state. The controller then installs the rules described in Section 4.2 and sends the packet on. Subsequent packets follow the installed rules, which carry a configurable idle timeout, resetting on traffic, and a hard timeout, after which the next packet triggers a fresh load-balancing decision (values in Appendix A). When a unit leaves a pool, its rules are removed. Two behaviours accompany the cost function: a warm-lease priority, which gives a newly admitted backend bounded priority for new connections during a configured window, and a slow-start ramp, in which a newly discovered backend is held back by a cost penalty that decays over the same window. Algorithm 4.2 summarises the first-packet path on both virtual addresses.

Algorithm 4.2 First-packet handling for a new connection.

1: the first packet of a new connection finds no flow rule
2: send the packet to the controller
3: select a unit from the eligible pool with the cost function
4: install the routing rule for the client and the unit (Section 4.2)
5: send the packet on, and later packets follow the rule

A request waits ahead of the edge service in three places, which the instrumentation separates. The first is the connection setup through the virtual service address. The second is the first packet's round trip to the controller and the installation of its rules. The third is the queue on the selected replica before the application reads the request. The first two are part of the steering path described here, and the third accrues on the serving side. The waiting time is read as a difference, user-visible latency minus the reported edge service time.

The controller keeps its state in memory, so a restart begins from an empty state and rederives the topology, the routing pools, and the per-node state from the network, flushing the stale flows and reinstalling the catch-all rule. The evaluated runs therefore exclude mid-run controller restarts.

CHAPTER 5

Experimental Results

This chapter reports the three campaigns in the order of the demand-to-capacity chain of Section 1.1: telemetry to decision (RQ1), decision to scaling action (RQ2), and ready backend to traffic (RQ3), closing with the synthesis across them.

5.1. Experimental Setup

This section defines the conditions under which the three research questions' experimental evaluations were executed.

5.1.1. Testbed and Instrumentation

The evaluation was performed on the platform of Chapters 3 and 4, deployed as a two-domain testbed on a single host. Each domain runs its own data plane, compute replicas, storage tier, telemetry aggregator, and SDN controller, and the two domains are joined by an emulated inter-domain link. Both domains are driven by the same open-loop driver of Section 4.1. The deployed components are described there, and the per-campaign caps, quotas, and parameter values are listed in Appendix A.

Each run is an independent experimental unit and starts from the same nodes and configuration. Before each run, the data set is generated from a fixed seed and loaded into the domain's replica-set primary, 3000 content items and 100 user profiles per domain, with the collection indexes created by the same setup. The setup exports a snapshot of the seeded items and profiles, and the driver draws its targets from it, so the offered demand is fixed by the run seed and does not depend on the live database. Between runs, the containers, the data state, and the controller state are reset.

During a run, the platform records the timeline of the demand-to-capacity chain end to end, from demand observation to the first request served by new capacity. The records behind these timestamps are described in Section 4.4, and every timing reported in this chapter is read from them. The rest of this section defines the workloads that drive these timelines, the outcomes extracted from them, and the design under which each campaign varied exactly one interface.

5.1.2. Demand Model and Campaign Workloads

A workload is a timed sequence of demand phases, and the phase that applies the demand shift under test is the demand episode. The demand model varies two axes, the request intensity over time and the request mix, which sets the tier an episode presses on. The mix is the per-request probability of each of six kinds (Table A.6), drawn independently for every request, so it sets how much of a phase's traffic presses on each tier. The load an episode places on a tier is its

press, and each campaign's demand episode is named after the tier it presses on. The weights are stated with each campaign's workload (Table 5.1).

All six kinds appear across the three campaign mixes. The telemetry campaign uses five (all but compute challenge), the scaling action campaign uses compute challenge, content lookup, and feed ranking, and the readiness admission campaign uses service pressure only.

The compute challenge gives the compute tier a press of its own. It performs a fixed number of hash rounds over a request-bound nonce with no storage access. Feed ranking loads both tiers, because it reads candidates before ranking them, and the cost of service pressure varies with the traffic already served. These differences are why the compute challenge carries the compute side of the scaling action campaign's mixed press.

Each phase fixes, for its duration, the offered request intensity (a per-client rate and an active-client fraction) and the request mix. Each campaign's workload was calibrated and locked before the campaign started, and every run stores the phase schedule it executed.

The phases around the episode have fixed roles across the campaigns. A baseline window opens every run at a low, steady load and establishes the healthy pre-shift operating point, a recovery gap lets the platform settle between the stressed and post-shift windows, a demand drop is the release window, in which the episode's capacity is scaled down and post-shift quality is compared with the baseline. Table 5.1 summarises each campaign's workload, with its rates and phase durations.

TABLE 5.1. Campaign workloads. Each campaign defines its own phase sequence, run in the order shown. The demand episode is the phase that applies the demand shift under test.

Campaign	Phase sequence	Demand episode
Telemetry Modes (RQ1)	Baseline (60 s) → load rise (180 s) → recovery gap (120 s) → demand drop (420 s) → idle tail (420 s)	Rate 1.2 per client, all clients active; mixed load: 30 % service pressure, 35 % content lookups, 15 % feed ranking, 10 % each of updates and aggregations
Bottleneck-Aware Scaling Action (RQ2)	Baseline (60 s) → mixed episode (600 s) → recovery gap (120 s) → demand drop (420 s) → idle tail (420 s), or baseline (30 s) → data-access episode (480 s) → recovery gap (60 s) → demand drop (360 s)	Mixed press: rate 1.15 per client, 20 % compute challenges and 80 % content lookups. Data-access press: rate 5.0 per client, 90 % content lookups and 10 % feed ranking. Compute-bound press: hash only, Section 5.3.
Readiness Admission Modes (RQ3)	Baseline (60 s) → compute-bound plateau (600 s) → recovery gap (120 s) → demand drop (420 s) → idle tail (420 s)	Rate 2.0 per client, pure service pressure; per-backend CPU quota 0.12

5.1.3. Metrics and Outcome Definitions

Every outcome is read from the recorded timeline of each run, and unless stated otherwise each run contributes the mean of its two domains. Outcomes are reported through offered

requests, completed requests, failed requests, timeouts (each recorded at the client-side cap of 300 s), and client-side cancellations and drops at phase end. Completion is reported on two bases, completed over non-abandoned requests and completed over offered requests, and latency percentiles use the served basis, where a served request is a completed request with a successful HTTP status. A latency component is a per-window mean over the campaign's demand episode, and each run is summarised by the median of its episode windows.

Usable capacity is the first moment at which new capacity serves traffic, measured from the anchor each campaign's interface defines: the onset of the demand shift where the telemetry mode varies, and the scaling decision where admission is under test. The remaining outcomes belong to the campaign that varies the interface they describe, and are defined with its results. Relief is the fall in the receiving tier's utilisation for a storage action and in the processing component for a compute action, while the added capacity carries served requests. Service recovery is how far completion and the two request-path components return towards their pre-shift levels during the episode. Control overhead is reported as the CPU utilisation, resident memory, and volume of control records of the controller processes.

5.1.4. Campaign Designs and Analysis

All three campaigns follow the same discipline of single-variable manipulation. Each campaign varies exactly one interface of the demand-to-capacity chain and holds the other two constant, so that the measured differences are attributable to the interface under test.

The treatments, primary outcomes, screening rules, and analysis tests of every campaign were fixed in writing before data collection.

Each campaign organises its runs so that every planned contrast is measured over matched demand. A seed fixes the demand of a run: which clients are active in each phase and each client's request stream, targets, payloads, and timing jitter. The runs are organised into blocks, a block holding one run of every treatment the campaign compares, sharing one seed, so the treatments entering a contrast meet identical demand. The execution order within a block is counterbalanced across blocks, so testbed drift cannot align with a single treatment.

Every campaign uses the same replication standard, seven seed-matched blocks per treatment, one run per treatment per block, and each block uses a distinct seed. The seed ranges are given in each campaign's design table. A campaign with multiple demand episodes runs one block set per episode over the same seven seeds. The readiness admission campaign pairs each fault run with its matched no-fault run inside the block. Seven seed-matched blocks per treatment were pre-registered, a size at which a fully separated contrast remains resolvable at the 0.05 significance level. Under complete separation between two treatment sets, the exact Mann–Whitney U test has a p-value floor of $2/\binom{2n}{n}$, 0.0006 at seven runs per treatment.

The run-level contrasts use the exact Mann–Whitney U test. A p-value is reported only where the runs of the two treatments overlap in range, and a complete separation is reported as such, since further replicates can only preserve or weaken it. The telemetry and readiness admission campaigns report Cliff's δ alongside, because a p-value alone does not show how

far apart the runs are. δ does not depend on the number of runs, a δ near 0 means the two treatments occupy the same range, and $\delta = \pm 1$ means complete separation.

Screening followed the pre-registered per-run checks, and the resulting exclusions, their effect on the analysis population, and any pre-registered criterion that was not met are reported with the campaign's results or in the configuration appendix. No exclusion rests on the treated outcome. The full design of each campaign is summarised at the start of its results section, the workload is in Table 5.1, and the replication is the standard fixed above.

5.2. Telemetry Modes (RQ1)

This campaign varies the first interface of the demand-to-capacity chain, the delivery of telemetry to the controller. It evaluated how the four telemetry modes change the way the controller scales the service when load rises, and reports what the controller saw, how much capacity it added and when the new capacity served traffic, and what users experienced.

The modes were compared over the same load rise (Section 5.1). Push (A) is the reference. Every summary reaches the controller as it closes. Delayed push (B) delivers every summary, 30 s late. Sampled push (D) delivers one summary in three, on time. Polling (C) delivers the same one-in-three share as sampled push, but the summary it returns had already closed when the read began, so its evidence arrives late as well as sparse (Section 3.4). Relative to the reference, one property changes at a time: delay without missing summaries (B), missing summaries without delay (D), and both together (C), so the cost of late telemetry is separated from the cost of missing telemetry. The pre-registered expectation was the ordering $\mathrm{A} < \mathrm{B} < \mathrm{C}$ on both usable capacity and the episode latency p95, with polling and sampled push expected to be indistinguishable from each other.

Table 5.2 summarises the design of the campaign, its workload is the telemetry row of Table 5.1.

TABLE 5.2. Design of the telemetry campaign (RQ1).

Item	Design
Treatment	Four telemetry modes over the otherwise identical platform: push (A); delayed push, the same summaries with a fixed 30 s lag (B); polling every 30 s (C); and sampled push, every third summary (D).
Workload	The telemetry row of Table 5.1: a baseline window, the load rise that is the demand episode, a recovery gap, a demand drop and an idle tail.
Replication	Seven seed-matched blocks, one run per mode per block (28 runs, seeds 3001–3007), counterbalanced order.
Test	Four pre-registered contrasts on the episode latency p95: delay (A–B, D–C) and missing evidence (A–D, B–C).
Expectations	Ordering $\mathrm{A} < \mathrm{B} < \mathrm{C}$ on usable capacity and the episode latency p95, with polling and sampled push expected to be indistinguishable; quality outside the load rise unaffected.

Under push and delayed push the controller received every completed summary, with the information age at decision being lowest under push and highest under delayed push. Under polling and sampled push it received about one third of them (Table 5.3).

TABLE 5.3. Telemetry modes: what the controller received and what users experienced. Delivered and information age are means over the seven replicates, with the observed range in parentheses; the other columns are medians. Information age is the time from the closing of the summary a decision used to the decision itself, and the metric definitions are in Section 5.1.3. No timeouts occurred in any mode.

Mode	Delivered (%)	Information age (s)	Usable capacity (s)	Episode p95 (s)	Cancelled (%)
push (A)	100.0	3.5 (1.2–7.5)	28.5 (25.1–32.9)	3.7 (2.8–5.2)	5.6
delayed push (B)	100.0	33.3 (30.6–38.4)	59.6 (55.1–61.5)	10.8 (5.0–65.3)	6.5
polling (C)	32.5–33.6	16.2 (13.8–19.8)	79.6 (73.7–81.8)	35.0 (27.0–39.6)	5.4
sampled push (D)	32.8–33.6	11.7 (5.3–15.9)	83.2 (75.1–88.1)	34.5 (31.4–37.6)	6.1

Every mode scaled up both tiers, and what differed was when the new capacity became usable (Table 5.3). First new capacity serving traffic came 28.5 s after the onset under push, and polling and sampled push took nearly three times as long, while delayed push landed close to its fixed 30 s lag. On usable capacity, A $<$ B and B $<$ C held in all seven blocks. C and D were not cleanly separated (D later by 3.6 s at the median, C $<$ D in five of seven blocks). The time to the first scale-up decision is descriptive only, because the telemetry mode sets when the triggering evidence reaches the controller. The number of actions is descriptive in the same way. Polling and sampled push ran 2 to 4 compute scale-ups per run, against 6, the whole dynamic pool, in push and delayed push. The trigger needs several qualifying summaries in a row, and a stream carrying one summary in three accumulates them more slowly, so the lossy modes added less as well as later.

The user-visible cost followed the scaling time (Table 5.4, Figure 5.1). With about two thirds of the summaries absent, the episode endpoint p95 rose roughly ninefold against push, and the registered A–D contrast separated completely ($\delta = -1.000$). Delay degraded the outcome in aggregate ($\delta = -0.959$, $p = 0.0012$), but not uniformly. Two of the seven delayed push runs collapsed to the level of polling and sampled push (65.3 s and 64.7 s), while the other five stayed at 5.0–12.6 s.

TABLE 5.4. Pre-registered primary contrasts on the episode endpoint latency p95 (served basis, per-run mean of the two domains). Cliff's δ is given with its 95 % bootstrap interval where the runs overlap, and the exact p-value only where the runs overlap, since complete separation at seven runs per mode has the floor stated in Section 5.1.4. The block count is how many seed-matched blocks followed the predicted order.

Contrast	Medians (s)	Blocks in predicted order	Separation
Delay A–B	3.7 vs 10.8	7/7	$\delta = -0.959$ [-1.000, -0.755]; $p = 0.0012$; not complete
Delay D–C	34.5 vs 35.0	2/7	$\delta = -0.265$ [-0.837, 0.388]; $p = 0.456$
Loss A–D	3.7 vs 34.5	7/7	complete, $\delta = -1.000$
Loss B–C	10.8 vs 35.0	5/7	$\delta = -0.429$ [-1.000, 0.429]; $p = 0.209$

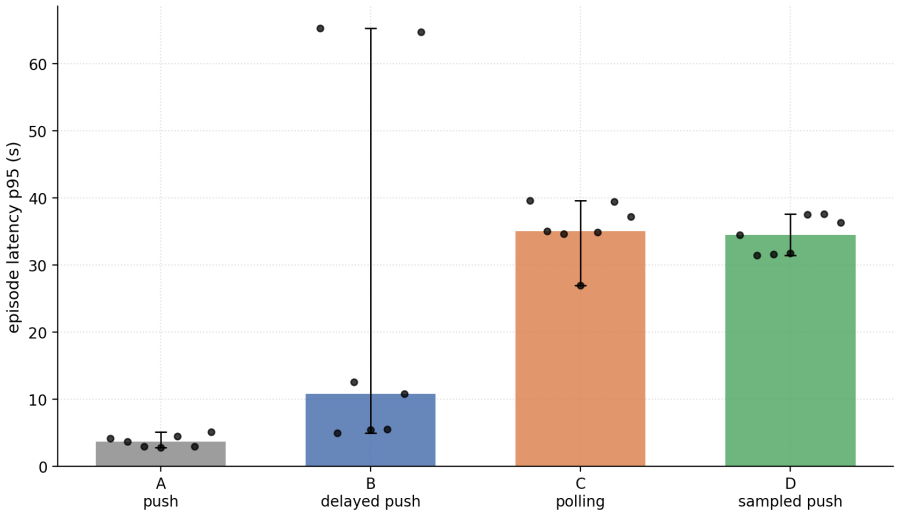


FIGURE 5.1. Episode endpoint latency p95 by telemetry mode (served basis; per-run mean of the two domains; seven replicates per mode). Bars show the median across replicates, whiskers the min–max range, and dots the individual runs; the two highest delayed push points are the replicates where the spare capacity of the nodes already running was consumed.

The spread of delayed push comes from the platform state that each run samples, not from the decision engine. Across all seven runs the first scale-up decision fell between 52.0 s and 60.4 s and usable capacity between 55.1 s and 61.5 s. What decides the outcome is whether the absorption margin of the nodes already running, their spare capacity while new capacity starts, covers the time they absorb the load alone: 25–33 s in A, 55–62 s in B, 74–82 s in C and 75–88 s in D. A slow run can exhaust that margin. Across runs of identical configuration the episode's median compute processing time varied sixfold (16–94 ms). A's absorption window was covered in every run, so the variance left no trace. C's and D's were exhausted in all seven. Delayed push sat at the boundary, covered in five runs and exhausted in the two that sampled a slow state.

One pre-registered expectation did not hold, and it is informative. The ordering broke at B–C because the two collapsed delayed push runs landed at C's level, so the two do not separate (exact $p = 0.209$). Polling and sampled push were indistinguishable ($p = 0.456$), as the design expected. What the campaign measured instead is two faults of different kinds. Missing summaries act robustly in every replicate, while delay acts conditionally on the margin it consumes.

The differences are confined to the load rise. Outside it, the four modes were indistinguishable, with baseline, recovery and demand-drop p95 at about 1.05–1.08 s, no timeouts inside the episode, and cancellation and drop shares flat at roughly 5–7 %.

5.3. Bottleneck-Aware Scaling Action (RQ2)

This campaign varies the second interface of the demand-to-capacity chain, the tier the scaling action serves. It evaluated what following the press buys over always scaling the same tier. Four scaling policies were compared under three demand episodes: no scaling, compute-only, storage-only, and bottleneck-aware, which scales compute and storage according to the pressing

demand. Each policy was run once per episode, over seven seed-matched blocks. The episodes differ in their request mix. The mixed press carries interactive requests with content lookups, so both tiers are loaded. The data-access press is read-heavy content serving, so the reads land on the storage tier. The compute-bound press is hash-only and issues no storage reads.

Each policy is a way to spend capacity. No scaling is the platform without elasticity. It shows what the press does alone and is the reference for every other policy. Compute-only is the usual reflex, the single elastic dimension of a plain container platform. Storage-only is the storage tier as the elastic dimension, the activation of a prepared reserve while the compute tier stays fixed. The bottleneck-aware policy is the platform under test. Both tiers can be scaled, the action follows the per-tier telemetry, and one tier is committed per action, so an episode that presses both tiers can be answered on both in sequence. A fixed policy can never scale the other tier.

Table 5.5 summarises the design of the campaign.

TABLE 5.5. Design of the scaling action campaign (RQ2).

Item	Design
Treatment	Four scaling policies under three demand episodes: no scaling, compute-only, storage-only, and bottleneck-aware, which commits one tier per action from the per-tier telemetry. Compute scale-up adds a replica. Storage scale-up activates a prepared reserve in the same domain.
Workload	The mixed and data-access presses of Table 5.1 and a compute-bound press, each run once per block under the policy under test.
Replication	Seven seed-matched blocks (seeds 42–48); each policy runs once per episode per block, with the policies paired within each seed.
Test	Pre-registered direction criteria, paired within each seed and assessed on direction alone (Section 5.1.4): four on the data press, five on the mixed press, and one on the compute-bound press. The medians and block counts are reported alongside; the registered test is direction alone, so no p-values or effect sizes are reported.
Expectations	Data press: the storage-only policy does better than no scaling and better than the compute-only policy on the data-access component and on relief, alone and as the first action of the bottleneck-aware policy. Mixed press: every policy that scales does better than no scaling, and the bottleneck-aware policy does better than each single-tier policy. Compute-bound press: the compute-capable policies move the press and storage-only does not.

With no scaling, the queue formed in the tier each request mix targets. The mixed press sat in the processing component and the data-access press in the storage tier, so each tier is bound in one episode and loaded without binding in the other (Table 5.6).

In the mixed press, the two policies that scaled the compute tier removed the processing component, and completion rose to about 99 % (Table 5.6, Figure 5.2). The data-access component fell with it, because a request's read no longer waits behind the processing queue. Storage-only relieved the tier it served, with the admitted members reporting activity, but the processing component stayed at its un-scaled level. Relief of a tier that does not bind did not reach users. The bottleneck-aware policy scaled storage first and compute after, matched compute-only on the processing component and on completion, and came in lower on the data-access component. The ordering was bottleneck-aware, compute-only, storage-only, no scaling, in every seed on

TABLE 5.6. Scaling policies by demand episode: the two request-path components, completion, storage utilisation, and the actions taken. Processing and data access are the components defined in Section 3.2.2, completion is completed requests over offered requests during the episode, and storage utilisation is the tier average over the episode. Values are medians across the seven seed-matched runs per policy. Compute scale-ups are per domain and run. Storage admissions count the activation of a prepared reserve and the preparation of its replacement, per domain and run. They are admission events, not concurrent members, and the concurrent count stays within the 6/6 cap. The figures show the per-run values.

Policy	Processing (ms)	Data access (ms)	Completion (%)	Storage (%)	Compute scale-ups / storage admissions
Mixed press					
no scaling	124	103	70.6	42	none
compute-only	10	10	98.4	52	3 compute scale-ups
storage-only	115	90	74.0	28	7 storage admissions
bottleneck-aware	10	6	98.9	30	7 storage admissions, 3 compute scale-ups
Data-access press					
no scaling	55	2075	77.5	98	none
compute-only	15	3615	82.1	99	1 compute scale-up
storage-only	21	66	84.7	43	6 storage admissions
bottleneck-aware	15	78	92.0	46	6 storage admissions, 1 compute scale-up

the data-access component, and on completion the two policies that scaled compute finished within a point of each other, well above the storage-only policy.

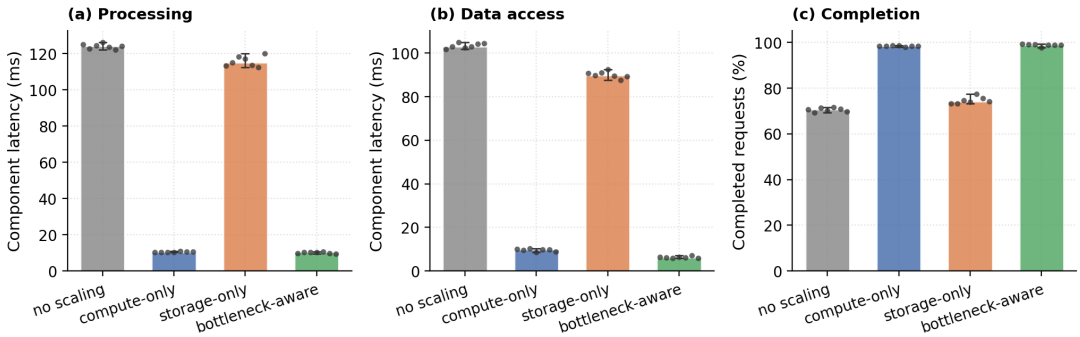


FIGURE 5.2. Mixed press: processing (left), data access (middle), and completion (right) by scaling policy. Bars are medians across the seven seed-matched runs, whiskers the min–max range, and dots the individual runs. Policy colours follow the chapter's fixed order.

The pattern follows the queue. The policy that removes the binding component wins the episode, and the mixed press loads both tiers, so a policy that scales one tier alone leaves the other unaddressed. All ten registered directions held in seven of seven blocks, and on the data-access component every block separated without overlap. The storage policy's gain was in relief more than in service, and the bottleneck-aware margin over compute-only was small. Both follow from the episode. The storage tier was loaded without binding, and the compute policy had already removed the only queue.

In the data-access press the queue was in the storage member, and storage-only was the matched policy. It activated the reserve in every run, the added members reported activity, and completion rose to about 85 %, taking the data-access component lowest of the three policies that scale. The bottleneck-aware policy scaled the storage reserve first and one compute unit after. It reached the highest completion in the episode, held the data-access component between the storage-only policy and no scaling, and brought the processing component to the level of the compute-only policy (Table 5.6, Figure 5.3).

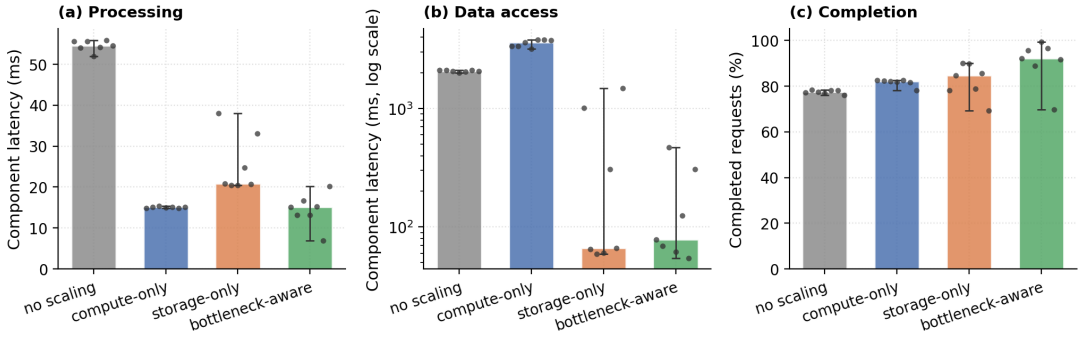


FIGURE 5.3. Data-access press: processing (left), data access (middle, logarithmic axis), and completion (right) by scaling policy. Bars are medians across the seven seed-matched runs, whiskers the min–max range, and dots the individual runs.

Compute-only removed the edge queue and pushed more requests at the un-scaled member. The data-access component rose above no scaling, and completion improved only marginally. The mismatched policy left the pressed tier worse than no scaling. The difference is the data path. The storage action adds a member and renews the connections that choose it, so reads that arrive afterwards can share the member, while the compute action shortens the path up to the queue and leaves the queue in place.

Under a compute-bound press (hash-only mix, 600 s, 0.23 requests per client per second), only the two compute-capable policies move the load: compute-only and bottleneck-aware complete 100 % of the offered requests at a p50 of 0.17–0.26 s and a p95 of 1.8–2.3 s, while storage-only and no scaling remain at 95–99 % completion, p50 2.2–2.7 s, and p95 8.0–22.2 s. The storage capability does not engage: the hash-only mix issues no storage reads, and no storage admission occurs in any run. Were the storage action forced, it would add members that receive no reads, spending capacity without relief.

Resource use separates the policies where service does not. On the mixed press the two policies that scaled compute spent their scale-ups on the tier that bound, while storage-only made more storage admissions and relieved a tier that did not, and its recovered delivery also raised the storage tier's utilisation. On the data-access press the two policies that scaled storage relieved the binding tier, while compute-only added one unit and left the press in place. The bottleneck-aware policy is the only one that spent on both tiers in both episodes, and it ended at or above the best single-tier policy on completion in both (Table 5.6, Figure 5.4). What reached users was decided by the tier a policy served, not by the number of actions it took.

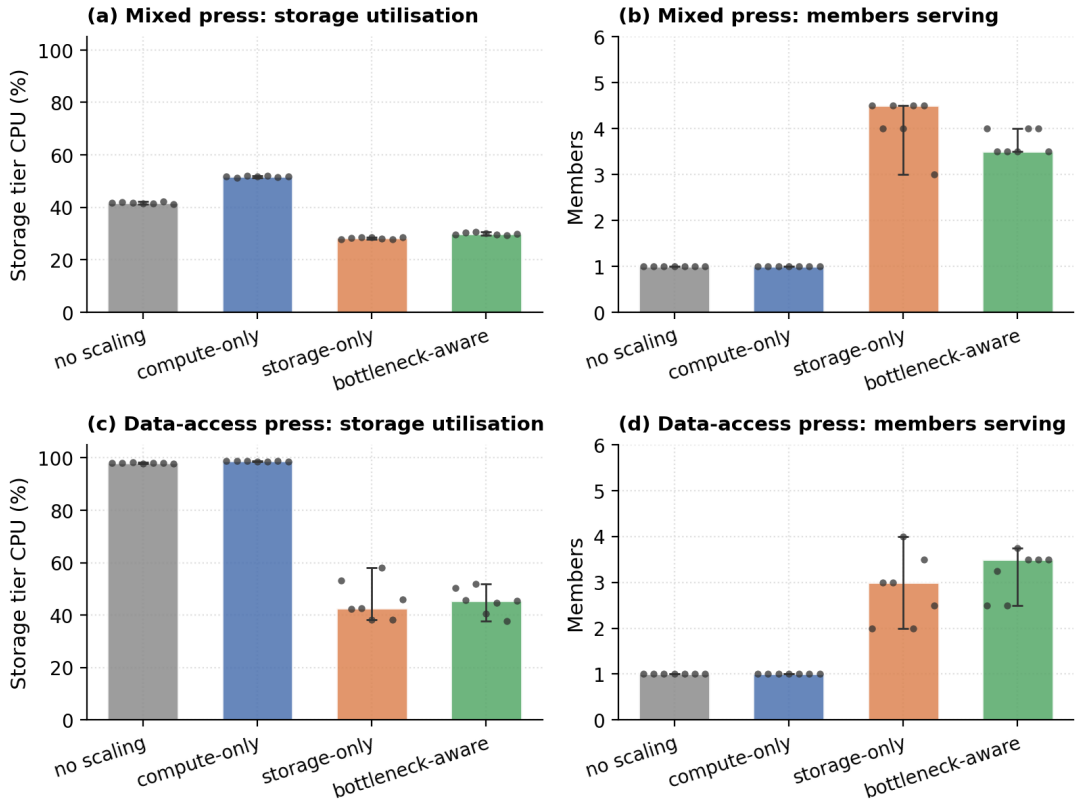


FIGURE 5.4. Relief and execution, by scaling policy: mixed press storage utilisation (a) and members serving (b); data-access press storage utilisation (c) and members serving (d). Bars are medians across the runs, whiskers the min–max range, and dots the individual runs.

Overall the bottleneck-aware policy gave the furthest service recovery in both episodes, and the matched single-tier policy came second in both. A policy that served a tier the episode did not press on produced no established user-visible benefit under the tested conditions, as storage-only did on the mixed press, or pushed more load onto the tier that was already pressed, as compute-only did on the data-access press.

5.4. Readiness Admission Modes (RQ3)

This campaign varies the third interface of the demand-to-capacity chain, the admission of a newly provisioned backend into the routing pool, that is, into the set of backends that can receive traffic. The backend announces that it can serve by sending a message, and the four modes differ in how much they trust it. In practice the message can be dropped on the way, or arrive before the backend can serve, so the modes are compared under three message conditions, each against its pre-registered expectation (Table 5.7); the plateau is the readiness row of Table 5.1.

The modes differ in what the message is trusted to do (Section 3.3.2). The 20 s wait and the 10 s period were calibrated in implementation testing. Both sit well inside the abandonment bound, and the wait is long enough that a message still in flight is not mistaken for a lost one.

TABLE 5.7. Design of the readiness admission campaign (RQ3).

Item	Design
Treatment	Readiness-admission coordination across the four admission modes: backend notification, notification with a fallback check (the control, recovering a lost message without verifying it), periodic polling, and notification with a confirmation check; the four modes are crossed with three message conditions.
Workload	One message condition per run: the message arrives intact, is lost, or arrives before the backend can serve, at a ten-second lead in the evaluated matrix.
Replication	Seven seed-matched blocks per mode and condition (intact and lost at seeds 5201–5207, premature at seeds 6301–6307, each fault block paired with a matched no-fault run), mode order rotated per block; the lost comparison uses the fixed six-block frame, with the added seventh block report-only. Admission timing uses separate intact-source sets (per-backend, higher-rate, relief, and poll-interval sensitivity), each with its own fixed workload and quota; samples are never pooled across sets.
Test	Exact Mann–Whitney U on the run-level values where a p-value is reported, and paired per-block direction criteria (Section 5.1.4).
Expectations	Intact: the modes that admit on the message admit earlier, even when a backend stalls at bind, the admission interval is bounded by the configured polling period, and the modes do not differ once admission occurs. Lost (reduced headroom): a decisive but bounded slow-share difference for backend notification, no redistribution under backend notification, recovery by the modes that check with relief completing, and the no-fault runs staying within its pre-registered healthy bounds. Premature: the modes that admit on the message are exposed while the modes that check hold their no-fault level.

Notification with a fallback check behaves like backend notification while the message arrives, because its fallback only fires when none does, so the priced contrast is between admitting on the message and checking for it. Backend notification admitted each backend essentially as soon as it could serve, periodic polling only at the next check, and every backend was admitted earlier under backend notification ($\delta = -1.000$). The full chain is in Table 5.8 and Figure 5.5.

Provisioning to the first successful response, across both start-up classes, was about six seconds shorter under backend notification (exact $p = 0.0005$), and relief reached the loaded tier about four seconds sooner (complete separation, $\delta = -1.000$; the set's standardised replicate reproduced the direction, earlier for every backend it measured). Once admitted, both modes served the first request in about a second, and the load left the existing nodes by the same amount under both (relief $p = 0.209$). The mode changes when the redistribution begins, not how much it moves. Periodic polling pays its polling granularity and nothing more. With the period widened from 10 s to 15 s (six runs), the admission interval moved from 9.62 s to 15.22 s and the time from the scaling decision to usable capacity from 11.45 s to 16.97 s, both by about the added five seconds (descriptive, Cliff's $\delta = -0.444$). The delay stayed inside the spare capacity of the existing nodes, the margin that the lost-message part of the campaign deliberately removes.

No user-visible cost of the seconds-level delay appeared in any tested healthy regime. Its backends reached the first successful response in a median of 4.0 s against 4.1 s under backend notification, a near-tie, and close to six seconds before periodic polling (10.0 s). The comparison covers only backends that bind in under a second. The stall of about ten seconds at socket bind occurs in the campaign's earlier runs and in none of its later ones, and the confirmation check's runs, which all come from the later period, contain no stalled backend.

TABLE 5.8. Admission-chain intervals and healthy-regime outcomes (intact message; backend notification versus periodic polling). Medians over each row's population: the readiness→admission row covers the recorded admission-timing set (12 runs, 47 backends), provisioning→first success pools both start-up classes, and scaling decision→usable capacity uses the higher-rate set. Exact Mann–Whitney p-values where the modes overlap.

Interval	Backend notification	Periodic polling	Test
Verified readiness→admission (s)	0.001	6.984	$\delta = -1.000$; earlier for every backend
Provisioning→first successful response (s)	11.27	17.70	exact $p = 0.0005$, $\delta = -0.594$
Scaling decision→usable capacity (s)	2.17	6.01	$\delta = -1.000$ (complete separation); standardised replicate 1.87 against 8.98 s
Admission→first routed request (s)	0.4–1.2	0.4–1.2	confirmatory; not separated
Relief of the existing nodes (per-run median, pp; seven runs per mode)	16.0	19.3	exact $p = 0.209$; not separated

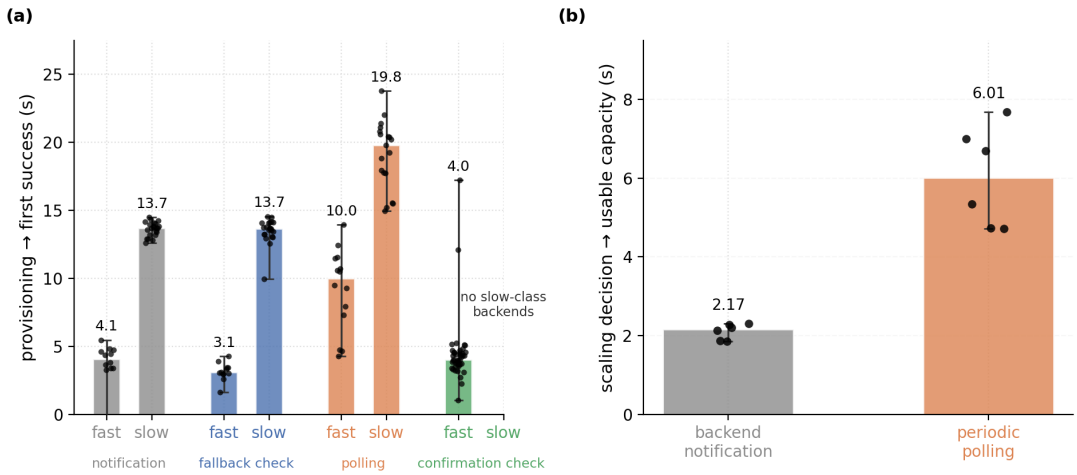


FIGURE 5.5. Admission-chain intervals under an intact message, for the four admission modes. (a) Provisioning to first successful response per backend (medians 4.1, 3.1, 10.0 and 4.0 s for backends that bind in under a second; 13.7, 13.7 and 19.8 s for the backends that met the stall at bind, none of them under the confirmation check). (b) Scaling decision to usable capacity per run (higher-rate set, six runs per mechanism). Dots are individual backends (a) and runs (b), bars and marks are medians, and whiskers span the min–max range.

When the message is lost, the comparison runs under a reduced per-backend CPU quota, to reduce the spare capacity and therefore the absorption margin. Backend notification admitted no backend in any loss run, with every candidate abandoned and torn down (32–35 per block; Table 5.9 and Figure 5.6). The modes that check recovered. Notification with a fallback check admitted after its fallback wait, about 21 s after provisioning, and periodic polling within one polling period, about four seconds sooner (medians 17.1 s against 21.4 s). In the premature grid's loss runs (seeds 6301–6303), periodic polling also recovered, landing $+1.3$ pp from its matched no-fault runs.

Under backend notification the slow share (completed or timed-out requests above one second) rose to 8.3–14.4 % over the plateau, concentrated in its first 150 s, where the difference against the recovering modes had a median of 28.0 pp (full plateau 7.64 pp, exact $p = 0.0040$). It stayed bounded at a 10.6 % plateau median, under the pre-registered ceiling of 25 %. The existing nodes kept serving throughout, so what the failure removed was the spare capacity, not the service.

TABLE 5.9. Admission modes under a lost message (fixed six-block frame, five assessable; the confirmation check covers its seven blocks). Admitted means visibility in the routing pool, with abandoned candidates in parentheses. Plateau slow is the slow share over the full plateau, plateau tail over its last 300 s, and a named block gives that block's own value.

Mode	When lost	Admitted (abandoned)	Plateau slow (%)	Plateau tail (%)
Backend notification	none; fallback inert	0 (32–35)	8.3–14.4	1.0–1.2, except b2: 11.4
Notification with a fallback check	20 s fallback check	4–6 (0)	1.9–3.6	$\leq$1.5
Periodic polling	10 s poll	4–5 (0)	2.0–5.0	$\leq$1.4, except b4: 4.1
Notification with a confirmation check	20 s absence fallback	6–7 (0)	1.1–4.1	0.7–3.7

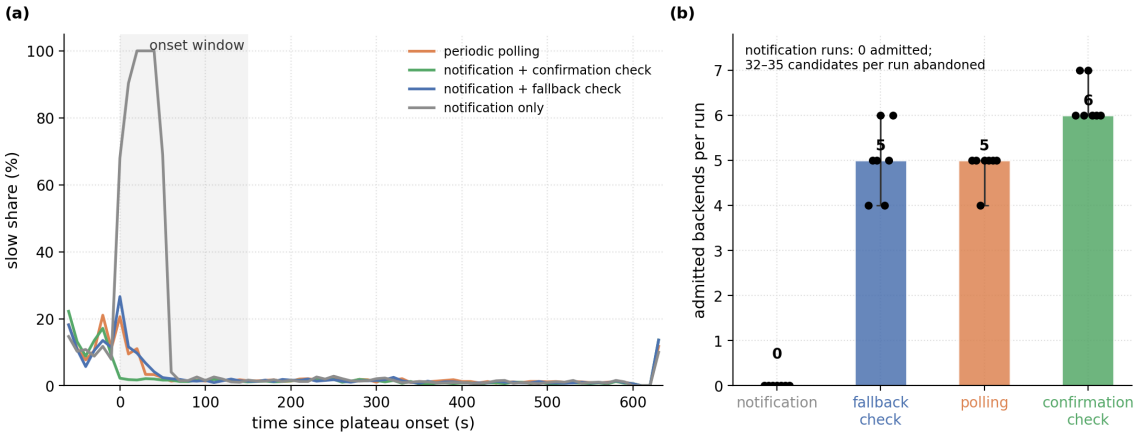


FIGURE 5.6. Under a lost message with reduced headroom, only the modes that check recover. (a) Slow share (completed or timed-out requests that took longer than 1 s), median across the seven seed-matched blocks per 10 s bin; the shaded band marks the first 150 s of the plateau. (b) Admitted backends per run (dots, one per block; bars, medians; whiskers, the min–max range). Backend notification admitted no backend and abandoned 32–35 candidates per run, while the other three modes admit through their own checks (medians 5, 5 and 6). The gated contrasts use each mode's assessable blocks (Table 5.9).

One qualifier bounds the reading. Two intact-run health-floor checks (blocks 3 and 5) fell outside their pre-registered bounds, so the criteria set for the lost condition were not all met together. The breaches sit on the intact side and do not enter the fault contrast. The block-4 exclusion rests on its pre-registered relief-completion check. Periodic polling's relief did not complete in that run, its plateau tail reading 4.08 % against the 2 pp bound (Table 5.9), so the block is not assessable. The confirmation check recovered through the same absence fallback, admitting 6–7 backends per run with none abandoned. Against its own no-fault partners it stayed within the 2 pp bound in every block (median change −0.4 pp), so the recovery carried no user cost.

Under a premature message at the evaluated lead of ten seconds, the expectation was that the modes that admit on the message would be exposed while the modes that check would absorb it (Table 5.10). Damage is the change, in percentage points, in the share of offered requests that fail, time out, or exceed one second, against a matched no-fault run. Backend notification and notification with a fallback check carried it (medians +14.4 and +11.7 pp) and stood +17.0 and +16.2 pp above periodic polling, positive in every block. These contrasts are medians of the per-pair differences, not differences of the per-mode medians in Table 5.10. Periodic polling stayed at its no-fault level. The expectation held, and the justification is the evidence each mode trusts. The fallback check reacts to a message that never arrives, not to one that arrives early, so it shares the exposure, while a check cannot pass before the backend can serve, so the timing of the message has nothing to act on. The confirmation check was immune in the same way (median −0.9 pp, six of seven pairs within the 2 pp bound) and admitted only once it could serve, reaching the first successful response 11.0 s after the message against 19.5 s under periodic polling. Backend notification was +16.9 pp more damaged than it, while the

confirmation check and periodic polling sit at a descriptive near-tie (-0.2 pp, three of seven pairs positive).

TABLE 5.10. User-visible damage of the admission modes under a premature message at the evaluated ten-second lead (seven seed-matched pairs per mode). Damage is the change, in percentage points, in the share of offered requests that fail, time out, or exceed one second, against a matched no-fault run; medians over the usable pairs (six under backend notification, seven under the other modes). A positive reading means the fault run's share was above that of its no-fault partner.

Mode	Median damage (pp)	Positive pairs	Within 2 pp
Backend notification	$+14.4$	6/6	0
Notification with a fallback check	$+11.7$	6/7	2
Periodic polling	-0.1	3/7	7/7
Notification with a confirmation check	-0.9	1/7	6/7

In sum, while the message was intact, the admission interval was bounded by the polling period and passed without user-visible cost. When the message was lost under reduced headroom, backend notification paid a bounded service-quality cost while the modes that check recovered. And when the message arrived early, the modes separated by what their evidence can see. A premature message cost the modes that admit on it while the modes that check priced it at nothing. Readiness-admission coordination is therefore at once a latency choice (about seven seconds of added delay while everything works) and a reliability choice, deciding whether a ready backend's capacity is delayed or lost when the message does not arrive, and whether a premature message reaches traffic before the backend can serve. Because admission fills the routing pool, the same choice sets what the load distribution can rely on when load has triggered the scaling. A lost message hides ready capacity from the pool. A premature message exposes capacity that cannot yet absorb traffic.

5.5. Discussion: Synthesis Across Interfaces

This section reads the three campaigns together, comparing the pattern each campaign revealed, classified per interface in Table 5.11 as robust, conditional, or not established.

TABLE 5.11. Synthesis of the three campaigns, classified by whether a finding held for every replicate entering the contrast (robust), depended on a condition the campaign varied or sampled (conditional), or could not be confirmed (not established). Every entry remains within its own campaign, and no quantity is combined across campaigns.

Interface	Robust	Conditional	Not established
Telemetry Modes (RQ1)	Loss of intermediate evidence. New capacity became usable nearly three times later than under push, the episode p95 penalty was about ninefold, and the A–D contrast separated completely ($\delta = -1.000$)	Delay: costly in proportion to the absorption margin it consumes, with two of seven delayed push runs collapsing to the level of polling and sampled push	Ordering B $<$ C on the episode p95, and sampled push worse than polling (D $>$ C): neither separated
Bottleneck-Aware Scaling Action (RQ2)	The storage-only policy under the data-access press, which relieved the pressed tier and reached the data-access wait; on the mixed press every policy that scales did better than no scaling, and the bottleneck-aware policy did better than each single-tier policy, with all ten registered directions holding in seven of seven blocks. On the compute-bound press, only the compute-capable policies moved the press, and the storage capability did not engage	Where the queue is. The storage action relieved a tier that was loaded without binding on the mixed press and produced no established user-visible benefit, while the compute-only policy left the pressed storage tier worse than no scaling on the data-access press	Why the size of the storage action's gain varies across runs. No user-visible advantage of the bottleneck-aware policy over the matched single-tier policy beyond completion under the tested conditions
Readiness Admission Modes (RQ3)	Loss without a check leaves the new capacity unused (zero admissions in every loss run), and a premature message damages the modes that admit on it (medians $+14.4$ and $+11.7$ pp) while the modes that check do not (medians -0.1 and -0.9 pp, within the 2 pp bound)	Both the size of the loss cost and the premature-message damage were measured under one condition each: the reduced headroom the campaign fixed, and a ten-second lead	Two no-fault health-floor breaches meant not all criteria for the lost condition were met together

Read across the campaigns, the measured effects share a shape. They appear where the service runs out of margin, not in the steady state. The telemetry modes were indistinguishable outside the load rise, and delay cost only when the absorption margin of the existing nodes ran out. The action improved user-visible quality only where it removed the queue the episode formed. The admission delay cost nothing while spare capacity remained, and the lost message consumed that margin under the reduced headroom the campaign fixed. The three interfaces decide how a demand shift is weathered, not how the service performs once it has passed.

The campaigns also separate failure modes that monitoring and scaling are often discussed as one. Missing summaries and late summaries are different faults. The first cost about nine times the episode tail with complete separation, and the second cost in proportion to the margin it consumed. In the action interface, execution and benefit are separate. The storage action relieved its tier on the mixed press, where that tier was loaded but not binding, and users gained little, while the compute action left the pressed tier worse than no scaling on the data-access press. In the admission interface, latency and reliability are separate, and one choice sets both at once. The mode fills the routing pool, so a lost message hides ready capacity from it and a premature message exposes capacity that cannot yet absorb traffic.

The interfaces also differ in the control records they leave behind, but not in the controller cost that carries them. Across the three campaigns the controller's memory stayed within a narrow band, at 66–77 MB in every configuration, and its CPU within 3.7–24 %. The delivery, check, and action records differed as designed (medians per configuration in Table A.4).

The limitations of the evaluation are detailed in Chapter 6.

CHAPTER 6

Conclusions and Future Work

This chapter answers the research questions and sets out the contributions, limitations, and future research.

6.1. Conclusions

This dissertation examined three interfaces of the demand-to-capacity chain and established, for each, which property of the crossing decides the outcome and under which conditions: telemetry consumption by the controller, the tier served by the scaling action, and the admission of a ready backend to traffic. Each interface was varied on its own, on one platform, in its own campaign. Table 6.1 summarises the answer to each research question.

TABLE 6.1. Answers to the research questions and the evidence that reports them. The classification of each finding within its own campaign is in Table 5.11.

RQ	Answer	Evidence
RQ1	Missing summaries drove the user-visible cost in every replicate. With one summary in three, new capacity became usable nearly three times later and the episode p95 rose about ninefold. Delay cost only where the absorption margin ran out, in two of seven delayed runs.	Section 5.2; Tables 5.3 and 5.4.
RQ2	The tier that served the press decided the user-visible outcome. On the compute-bound press, only the compute-capable policies moved the press, and the storage capability did not engage. Storage-only relieved the data-access press in every seed, while compute-only left the pressed tier worse than no scaling. On the mixed press, storage-only relieved a tier loaded without binding and produced no established user-visible benefit. The bottleneck-aware policy gave the highest completion under all three presses and spent on both tiers in the mixed and data-access presses, but no advantage over the matched policy beyond completion was established.	Section 5.3; Table 5.6.
RQ3	One configuration set latency and reliability at once. With the message intact, backend notification admitted about seven seconds earlier than polling at no user cost for fast-binding backends. With the message lost, notification admitted nothing while the modes that check recovered. With a premature message, only the modes that admit on it were damaged; the confirmation check also reached first success soonest.	Section 5.4; Tables 5.8, 5.9, and 5.10.

Read together, the three answers describe one mechanism, developed in Section 5.5. Each interface spends a margin that the running service holds. Telemetry loss stretches the window in which the current capacity has to cope alone, an action that serves the wrong tier adds capacity where no margin is being consumed, and an admission that trusts the readiness message withholds ready capacity when the message is lost and exposes unusable capacity when it arrives early. The user is affected only where the margin runs out, which is why the interfaces act on the transition rather than on the steady state. No aggregate cost along the chain is claimed, because

the campaigns vary one interface at a time under different workloads and their segments are not additive.

The motivations of Section 1.1 close as findings or explicit non-claims. The storage tier behaved as a live capacity that must follow demand, and the storage action reached users only when the press bound that tier (RQ2). Scale-up alone did not guarantee usable capacity instantly. The telemetry mode set when new capacity became usable, and the admission mode set when a ready backend began serving (RQ1 and RQ3). The crossings differed in more than latency: missing evidence, a mismatched tier, and an unverified readiness message each produced a distinct failure (RQ1, RQ2, and RQ3). The replication bandwidth consumed by a membership change was not metered.

6.2. Research Contributions

The contributions stated in Section 1.5 are supported as follows.

(1) *Telemetry modes (RQ1).* The four modes were compared on an otherwise identical platform, separating the two faults they carry. Missing intermediate evidence cost in every replicate relative to push, while a complete but delayed stream cost conditionally, in proportion to the absorption margin it consumed (Section 5.2).

(2) *Bottleneck-aware scaling action (RQ2).* Scaling the tier under pressure was compared with two fixed single-tier policies and with no scaling under three demand episodes. The matched action relieved the pressed tier and reached users, a mismatched action produced no established user-visible benefit, and the bottleneck-aware policy gave the highest completion under all three episodes and was the only one that spent on both tiers in the mixed and data-access presses (Section 5.3).

(3) *Readiness admission (RQ3).* The four admission modes were crossed with three message conditions, measuring the admission step that the placement literature leaves outside its scope: a bounded delay while the platform is healthy, an all-or-nothing loss when the message is dropped without a check, and damage confined to the modes that admit on a premature message (Section 5.4).

(4) *Comparative synthesis.* The three interfaces were synthesised under one classification, robust, conditional, or not established within each campaign, with no quantity pooled across campaigns (Table 5.11).

6.3. Limitations

The architecture and the experiments share one trade-off. To vary an interface in isolation, telemetry consumption, the scaling decision, and routing had to run in one controller process, on one host, under one configuration of everything else. That choice is what makes the measured effects attributable to the interface under test, and it is also what bounds them. The limitations below follow from it.

The first cost is that the platform has no external baseline. Because the three functions are co-located, the evaluation compares configurations of one controller process per domain. It does not compare against an orchestration in which monitoring, autoscaling, and load balancing

run as separate processes or on separate nodes, so the cost of that separation is not measured, and the results cannot be read as a comparison against a conventional orchestrator. Co-location also removes distribution from the deployment. The testbed is a single host with two emulated domains, the control plane is host-local, and the data path is single-site, with no introduced latency or jitter between the controller and the resources it manages. No multi-site or wide-area claim is made, and the compute tier is a set of Docker containers rather than a cluster orchestrator, so scheduling, image distribution, and multi-node placement were not evaluated. Finally, co-location concentrates risk. The controller keeps its state in memory and has no leader election or failover, so a restart loses the routing-pool state until it is re-derived from the network. Its packet path runs inline in the OpenFlow event loop, so throughput under high connection churn was not characterised, and controller failure, recovery, peer-state expiry, and secure multi-domain control were not evaluated.

The second cost is in what the routing layer can observe. The routing pool is rebuilt on every topology tick rather than atomically on registration, so a backend can be recorded as admitted before it is selectable, and a draining backend can remain selectable until the next rebuild. At a one-second tick, the admission-to-first-request interval carries an up-to-one-second boundary. That boundary limits the precision of the admission timestamp, not the reported contrasts, which are several seconds apart, but it means the admission instant is a registration instant and not a routability instant. Routing is also connection-level, so an existing flow keeps its backend until it expires or is renewed, and a scaling action cannot redistribute traffic already in flight.

The third cost is in what the storage action can be asked to do. MongoDB accepts writes only on the replica-set primary, so storage scale-up adds read capacity and the evaluation covers read-bound storage pressure. The action activates a reserve prepared before the reaction path, so the measured action excludes cold member creation and initial synchronisation, and a deployment that creates a member on demand would see a longer and more variable action. The action is a membership change in the domain's own replica set, so cross-domain placement and the Tier 0 and Tier 1 data paths were held constant. Holding the other interfaces constant has a cost of its own. Each campaign varies one interface and fixes the other two, so the effects are measured in isolation and their interactions are not measured.

Within these bounds the trade-off holds. Each interface was varied alone over seed-matched blocks, on a platform whose other two interfaces and whose demand were fixed, and that is what allows an effect to be attributed to the interface under test. The bounds mark where the findings stop rather than where they weaken, and each opens a direction in Section 6.4.

6.4. Future Research Directions

Each direction below follows from one of the bounds set out in Section 6.3.

Compound treatments across interfaces. The campaigns vary one interface and fix the other two, so the effects are measured apart. An injection that delays several handovers at once, with a configurable delay at each, would test whether the effects compose.

An external baseline. With the three functions co-located, the platform has no separate-process orchestration to compare against. Deploying monitoring, autoscaling, and routing as

separate processes and repeating a campaign would measure the cost of that separation. Repeating the telemetry campaign over off-the-shelf observability components, such as an OpenTelemetry collector over a message broker, would show whether the measured effects are properties of the modes or of this implementation, and varying the window size in the same setting would trade freshness against noise.

Admission as routability. Admission is recorded at registration while the routing pool follows the topology tick, and an existing flow keeps its backend until it expires or is renewed. Updating the pool atomically on registration, and rebalancing in-flight connections when a scaling action fires, would make the admission timestamp a routability timestamp and let a scaling action reach traffic that is already running.

Storage placement and tiering. Only the Tier 2 membership change in the domain's own replica set was evaluated, and only with a reserve prepared before the reaction path. Evaluating Tier 0 and Tier 1, adding cold member creation to the reaction path, and instrumenting replica-sync bandwidth and the transient cost of a membership change would characterise the storage side of the chain and extend the storage action beyond read capacity.

Action policy beyond the operating envelope. The storage action was evaluated under read-bound pressure. A write-bound press would test the other side of the primary-only limit, a wrong-tier action would price its user cost, and regime shifts beyond the tested demand would show where a residual benefit is expected.

Scale, distribution, and realism. A larger deployment across separate hosts or sites, real traffic traces, and ML-driven threshold adaptation would test how far the findings extend beyond a single-host testbed. Controller redundancy, peer-state expiry, and secure multi-domain control would remove the control-plane single point of failure. A programmable data plane such as P4 would move steering and telemetry into the network, at the cost of reproducing the connection-tracking behaviour in-switch.

References

[1] K. L. Bogdanov, W. Reda, G. Q. Maguire Jr., D. Kostić, and M. Canini, "Fast and accurate load balancing for geo-distributed storage systems," in *Proceedings of the ACM Symposium on Cloud Computing*, Carlsbad CA USA, Oct. 2018, pp. 386–400. DOI: 10.1145/3267809.3267820 [Online]. Available: https://doi.org/10.1145/3267809.3267820

[2] V. Podolskiy, A. Jindal, and M. Gerndt, "Iaas reactive autoscaling performance challenges," in *2018 IEEE 11th International Conference on Cloud Computing (CLOUD)*, San Francisco, CA, USA, Jul. 2018, pp. 954–957. DOI: 10.1109/CLOUD.2018.00144 [Online]. Available: https://doi.org/10.1109/CLOUD.2018.00144

[3] International Telecommunication Union, "Facts and figures 2025: Internet traffic," ITU, Oct. 2025. Accessed: Jul. 13, 2026. [Online]. Available: https://www.itu.int/itu-d/reports/statistics/2025/10/15/ff25-internet-traffic/

[4] D. Gurung, S. M. Z. U. Rashid, Z. u. Abdeen, and S. Rath, "Cloud revolution: Tracing the origins and rise of cloud computing," in *2026 IEEE 16th Annual Computing and Communication Workshop and Conference (CCWC)*, Las Vegas, NV, USA, Jan. 2026, pp. 1100–1106. DOI: 10.1109/CCWC67433.2026.11393790 [Online]. Available: https://doi.org/10.1109/CCWC67433.2026.11393790

[5] T. Gaudiaut, *Mobile accounts for nearly 60 percent of web traffic*, Statista Chart of the Day, Data from StatCounter. Licensed under CC BY-ND 3.0, Mar. 2026. Accessed: Jul. 19, 2026. [Online]. Available: https://www.statista.com/chart/35951/share-of-mobile-device-website-traffic-worldwide-since-2010/

[6] K. Cao, Y. Liu, G. Meng, and Q. Sun, "An overview on edge computing research," *IEEE Access*, vol. 8, pp. 85 714–85 728, 2020. DOI: 10.1109/ACCESS.2020.2991734 [Online]. Available: https://doi.org/10.1109/ACCESS.2020.2991734

[7] M. Satyanarayanan, "The emergence of edge computing," *Computer*, vol. 50, no. 1, pp. 30–39, Jan. 2017. DOI: 10.1109/MC.2017.9 [Online]. Available: https://doi.org/10.1109/MC.2017.9

[8] Q. Luo, S. Hu, C. Li, G. Li, and W. Shi, "Resource scheduling in edge computing: A survey," *IEEE Communications Surveys & Tutorials*, vol. 23, no. 4, pp. 2131–2165, 2021. DOI: 10.1109/COMST.2021.3106401 [Online]. Available: https://doi.org/10.1109/COMST.2021.3106401

[9] C. Qu, R. N. Calheiros, and R. Buyya, "Auto-scaling web applications in clouds: A taxonomy and survey," *ACM Computing Surveys*, vol. 51, no. 4, pp. 1–33, Jul. 2018. DOI: 10.1145/3148149 [Online]. Available: https://doi.org/10.1145/3148149

[10] A. Llorens-Carrodeguas, I. Leyva-Pupo, C. Cervelló-Pastor, L. Piñeiro, and S. Siddiqui, "An sdn-based solution for horizontal auto-scaling and load balancing of transparent vnf clusters," *Sensors*, vol. 21, no. 24, p. 8283, Dec. 2021. DOI: 10.3390/s21248283 [Online]. Available: https://doi.org/10.3390/s21248283

[11] N. Yaseen, "From counters to telemetry: A survey of programmable network-wide monitoring," *Network*, vol. 5, no. 3, p. 38, Sep. 2025. DOI: 10.3390/network5030038 [Online]. Available: https://doi.org/10.3390/network5030038

[12] A. Y. Nikravesh, S. A. Ajila, and C.-H. Lung, "The impact of database layer on auto-scaling decisions in a 3-tier web services cloud resource provisioning," in *2017 IEEE 41st Annual*

Computer Software and Applications Conference (COMPSAC), Turin, Jul. 2017, pp. 401–406. DOI: 10.1109/COMPSAC.2017.235 [Online]. Available: https://doi.org/10.1109/COMPSAC.2017.235

[13] T. Dreibholz and S. Mazumdar, "Towards a lightweight task scheduling framework for cloud and edge platform," *Internet of Things*, vol. 21, p. 100 651, Apr. 2023. DOI: 10.1016/j.iot.2022.100651 [Online]. Available: https://doi.org/10.1016/j.iot.2022.100651

[14] A. R. Hevner, S. T. March, J. Park, and S. Ram, "Design science in information systems research," *MIS Quarterly*, vol. 28, no. 1, pp. 75–106, Mar. 2004. DOI: 10.2307/25148625 [Online]. Available: https://doi.org/10.2307/25148625

[15] J. vom Brocke, A. Hevner, and A. Maedche, "Introduction to design science research," in *Progress in IS*, 2020, pp. 1–13, ISBN: 9783030467807. DOI: 10.1007/978-3-030-46781-4_1 [Online]. Available: https://doi.org/10.1007/978-3-030-46781-4_1

[16] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, "A design science research methodology for information systems research," *Journal of Management Information Systems*, vol. 24, no. 3, pp. 45–77, Dec. 2007. DOI: 10.2753/MIS0742-1222240302 [Online]. Available: https://doi.org/10.2753/MIS0742-1222240302

[17] F. Liu, G. Tang, Y. Li, Z. Cai, X. Zhang, and T. Zhou, "A survey on edge computing systems and tools," *Proceedings of the IEEE*, vol. 107, no. 8, pp. 1537–1562, 2019. DOI: 10.1109/JPROC.2019.2920341 [Online]. Available: https://doi.org/10.1109/JPROC.2019.2920341

[18] A. C. Baktir, A. Ozgovde, and C. Ersoy, "How can edge computing benefit from software-defined networking: A survey, use cases, and future directions," *IEEE Communications Surveys & Tutorials*, vol. 19, no. 4, pp. 2359–2391, 2017. DOI: 10.1109/COMST.2017.2717482 [Online]. Available: https://doi.org/10.1109/COMST.2017.2717482

[19] Q. Waseem, W. Isni Sofiah Wan Din, A. Aminuddin, M. Hussain Mohammed, and R. F. Alfa Aziza, "Software-defined networking (sdn): A review," in *2022 5th International Conference on Information and Communications Technology (ICOIACT)*, Yogyakarta, Indonesia, Aug. 2022, pp. 30–35. DOI: 10.1109/ICOIACT55506.2022.9972067 [Online]. Available: https://doi.org/10.1109/ICOIACT55506.2022.9972067

[20] A. Liatifis, P. Sarigiannidis, V. Argyriou, and T. Lagkas, "Advancing sdn from openflow to p4: A survey," *ACM Computing Surveys*, vol. 55, no. 9, pp. 1–37, Jan. 2023. DOI: 10.1145/3556973 [Online]. Available: https://doi.org/10.1145/3556973

[21] MongoDB, Inc., *Replication*, MongoDB Manual, 2026. Accessed: Sep. 13, 2026. [Online]. Available: https://www.mongodb.com/docs/manual/replication/

[22] C.-K. Huang and G. Pierre, "Adappf: Self-adaptive scrape interval for monitoring in geo-distributed cluster federations," in *2023 IEEE Symposium on Computers and Communications (ISCC)*, Gammarth, Tunisia, Jul. 2023, pp. 417–423. DOI: 10.1109/ISCC58397.2023.10218080 [Online]. Available: https://doi.org/10.1109/ISCC58397.2023.10218080

[23] M.-H. Hung, C.-C. Teng, C.-P. Chuang, C.-S. Hsu, J.-W. Gong, and M.-C. Chen, "A sdn controller monitoring architecture for 5g backhaul networks," in *2022 23rd Asia-Pacific Network Operations and Management Symposium (APNOMS)*, Takamatsu, Japan, Sep. 2022, pp. 1–4. DOI: 10.23919/APNOMS56106.2022.9919988 [Online]. Available: https://doi.org/10.23919/APNOMS56106.2022.9919988

[24] M. Usman, S. Ferlin, A. Brunstrom, and J. Taheri, "A survey on observability of distributed edge & container-based microservices," *IEEE Access*, vol. 10, pp. 86 904–86 919, 2022. DOI: 10.1109/ACCESS.2022.3193102 [Online]. Available: https://doi.org/10.1109/ACCESS.2022.3193102

[25] H. Zhou and C. H. Yong, "Implement hpa for nginx service using custom metrics under kubernetes framework," *IEEE Access*, vol. 12, pp. 189 722–189 734, 2024. DOI: 10.1109/ACCESS.2024.3509876 [Online]. Available: https://doi.org/10.1109/ACCESS.2024.3509876

[26] M. I. S. El Mrabet, A. Malki, and M. Malki, "A comprehensive survey and taxonomy of autoscaling techniques in cloud and edge environments," in *2025 Fourth International Conference on Theoretical and Applicative Aspects of Computer Science (ICTAACS)*, Skikda, Algeria, Dec. 2025, pp. 1–8. DOI: 10.1109/ICTAACS69003.2025.11399335 [Online]. Available: https://doi.org/10.1109/ICTAACS69003.2025.11399335

[27] S. Taherizadeh and V. Stankovski, "Auto-scaling applications in edge computing," in *Proceedings of the International Conference on Big Data and Internet of Thing*, London United Kingdom, Dec. 2017, pp. 158–163. DOI: 10.1145/3175684.3175709 [Online]. Available: https://doi.org/10.1145/3175684.3175709

[28] L. Toka, G. Dobreff, B. Fodor, and B. Sonkoly, "Machine learning-based scaling management for kubernetes edge clusters," *IEEE Transactions on Network and Service Management*, vol. 18, no. 1, pp. 958–972, Mar. 2021. DOI: 10.1109/TNSM.2021.3052837 [Online]. Available: https://doi.org/10.1109/TNSM.2021.3052837

[29] L. Ju, P. Singh, and S. Toor, "Proactive autoscaling for edge computing systems with kubernetes," in *Proceedings of the 14th IEEE/ACM International Conference on Utility and Cloud Computing Companion*, Leicester United Kingdom, Dec. 2021, pp. 1–8. DOI: 10.1145/3492323.3495588 [Online]. Available: https://doi.org/10.1145/3492323.3495588

[30] K. Cheng, S. Zhang, C. Tu, X. Shi, Z. Yin, S. Lu, Y. Liang, and Q. Gu, "Proscale: Proactive autoscaling for microservice with time-varying workload at the edge," *IEEE Transactions on Parallel and Distributed Systems*, vol. 34, no. 4, pp. 1294–1312, Apr. 2023. DOI: 10.1109/TPDS.2023.3238429 [Online]. Available: https://doi.org/10.1109/TPDS.2023.3238429

[31] J. Dogani, R. Namvar, and F. Khunjush, "Auto-scaling techniques in container-based cloud and edge/fog computing: Taxonomy and survey," *Computer Communications*, vol. 209, pp. 120–150, Sep. 2023. DOI: 10.1016/j.comcom.2023.06.010 [Online]. Available: https://doi.org/10.1016/j.comcom.2023.06.010

[32] S. Xie, J. Wang, B. Li, Z. Zhang, D. Li, and P. C. K. Hung, "Pbscaler: A bottleneck-aware autoscaling framework for microservice-based applications," *IEEE Transactions on Services Computing*, vol. 17, no. 2, pp. 604–616, Mar. 2024. DOI: 10.1109/TSC.2024.3376202 [Online]. Available: https://doi.org/10.1109/TSC.2024.3376202

[33] C. Li, J. Bai, Y. Ge, and Y. Luo, "Heterogeneity-aware elastic provisioning in cloud-assisted edge computing systems," *Future Generation Computer Systems*, vol. 112, pp. 1106–1121, Nov. 2020. DOI: 10.1016/j.future.2020.06.022 [Online]. Available: https://doi.org/10.1016/j.future.2020.06.022

[34] M. R. Belgaum, S. Musa, M. M. Alam, and M. M. Su'ud, "A systematic review of load balancing techniques in software-defined networking," *IEEE Access*, vol. 8, pp. 98 612–98 636, 2020. DOI: 10.1109/ACCESS.2020.2995849 [Online]. Available: https://doi.org/10.1109/ACCESS.2020.2995849

[35] A. A. Neghabi, N. Jafari Navimipour, M. Hosseinzadeh, and A. Rezaee, "Load balancing mechanisms in the software defined networks: A systematic and comprehensive review of the literature," *IEEE Access*, vol. 6, pp. 14 159–14 178, 2018. DOI: 10.1109/ACCESS.2018.2805842 [Online]. Available: https://doi.org/10.1109/ACCESS.2018.2805842

[36] A. A. Abdelltif, E. Ahmed, A. T. Fong, A. Gani, and M. Imran, "Sdn-based load balancing service for cloud servers," *IEEE Communications Magazine*, vol. 56, no. 8, pp. 106–111,

Aug. 2018. DOI: 10.1109/MCOM.2018.1701016 [Online]. Available: https://doi.org/10.1109/MCOM.2018.1701016

[37] A. Wang, Z. Zha, Y. Guo, and S. Chen, "Software-defined networking enhanced edge computing: A network-centric survey," *Proceedings of the IEEE*, vol. 107, no. 8, pp. 1500–1519, Aug. 2019. DOI: 10.1109/JPROC.2019.2924377 [Online]. Available: https://doi.org/10.1109/JPROC.2019.2924377

[38] H. Peng, Q. Ye, and X. S. Shen, "Sdn-based resource management for autonomous vehicular networks: A multi-access edge computing approach," *IEEE Wireless Communications*, vol. 26, no. 4, pp. 156–162, Aug. 2019. DOI: 10.1109/MWC.2019.1800371 [Online]. Available: https://doi.org/10.1109/MWC.2019.1800371

[39] A. Nain, S. Sheikh, M. Shahid, and R. Malik, "Resource optimization in edge and sdn-based edge computing: A comprehensive study," *Cluster Computing*, vol. 27, no. 5, pp. 5517–5545, Feb. 2024. DOI: 10.1007/s10586-023-04256-8 [Online]. Available: https://doi.org/10.1007/s10586-023-04256-8

[40] E. Saurez, H. Gupta, A. Daglis, and U. Ramachandran, "Oneedge: An efficient control plane for geo-distributed infrastructures," in *Proceedings of the ACM Symposium on Cloud Computing*, Seattle WA USA, Nov. 2021, pp. 182–196. DOI: 10.1145/3472883.3487008 [Online]. Available: https://doi.org/10.1145/3472883.3487008

[41] M. A. Tamiru, G. Pierre, J. Tordsson, and E. Elmroth, "Mck8s: An orchestration platform for geo-distributed multi-cluster environments," in *2021 International Conference on Computer Communications and Networks (ICCCN)*, Athens, Greece, Jul. 2021, pp. 1–10. DOI: 10.1109/ICCCN52240.2021.9522318 [Online]. Available: https://doi.org/10.1109/ICCCN52240.2021.9522318

[42] H. Tabatabaee Malazi, S. R. Chaudhry, A. Kazmi, A. Palade, C. Cabrera, G. White, and S. Clarke, "Dynamic service placement in multi-access edge computing: A systematic literature review," *IEEE Access*, vol. 10, pp. 32 639–32 688, 2022. DOI: 10.1109/ACCESS.2022.3160738 [Online]. Available: https://doi.org/10.1109/ACCESS.2022.3160738

[43] B. Schroeder, A. Wierman, and M. Harchol-Balter, "Open versus closed: A cautionary tale," in *Proceedings of the 3rd Symposium on Networked Systems Design and Implementation (NSDI 2006)*, San Jose, CA, USA: USENIX Association, May 2006, pp. 239–252. Accessed: Aug. 4, 2026. [Online]. Available: https://www.usenix.org/conference/nsdi-2006/open-versus-closed-cautionary-tale

[44] Oracle Corporation, *Mysql adminapi*, MySQL Shell User Guide, 2026. Accessed: Oct. 11, 2026. [Online]. Available: https://dev.mysql.com/doc/mysql-shell/en/admin-api-userguide.html

[45] Oracle Corporation, *Innodb cluster*, MySQL 8.0 Reference Manual, 2026. Accessed: Oct. 11, 2026. [Online]. Available: https://dev.mysql.com/doc/refman/8.0/en/mysql-innodb-cluster-introduction.html

[46] PostgreSQL Global Development Group, *Log-shipping standby servers*, PostgreSQL Documentation, 2026. Accessed: Oct. 11, 2026. [Online]. Available: https://www.postgresql.org/docs/current/warm-standby.html

APPENDIX A

Campaign Configuration Values

This appendix lists the concrete configuration values of the three campaigns reported in Chapter 5, complementing the design summaries given at the start of each results section. The values are transcribed from the campaign configuration records; every run additionally stored the effective configuration it executed (`controller_env_snapshot.env`, `aggregator_env_snapshot.env`) and its phase schedule (`phases_snapshot.json`), so the per-run values are recoverable from the run artefacts. Where a campaign spans several demand regimes or evidence sets, each column states the value for its regime or set. Table A.1 lists the container limits, flow timeouts and client bounds, Table A.2 the scaling-policy parameters, Table A.3 the load-balancing weights and Table A.5 the implementation knobs. The control-plane cost of each configuration is reported with the synthesis in Section 5.5.

Every stored phase schedule follows the anatomy of Figure A.1, which draws the phases of a run and singles out the demand episode among them.

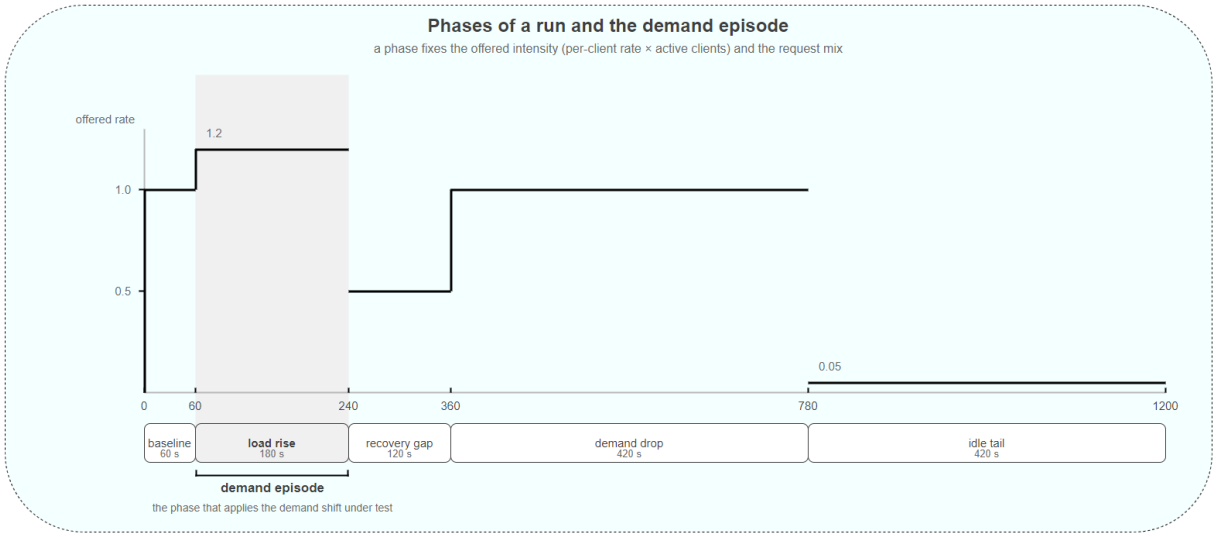


FIGURE A.1. Phases of a run, shown for the telemetry campaign. The phase that applies the demand shift under test is the demand episode, and the outcomes are read over it. The surrounding phases have fixed roles: the baseline window establishes the healthy pre-shift operating point, the recovery gap lets the platform settle, the demand drop is the release window in which the episode's capacity is scaled down, and the idle tail, where a campaign uses one, lets the release complete. Durations, intensities, and mixes differ per campaign and are listed in Table 5.1.

TABLE A.1. Container resource limits, flow timeouts, and client bounds per campaign.

Parameter	Telemetry Modes (RQ1)	Bottleneck-Aware Scaling Action (RQ2)	Readiness Admission Modes (RQ3)
Compute replica CPU quota (vCPU)	0.25	0.15 (compute-bound and mixed presses) / 1.20 (data-access press)	0.12 (fault runs); 0.15 / 0.30 (intact-source sets)
Compute replica memory limit	256 MB	384 MB (256 MB for the compute-bound block)	256 MB
Storage-member CPU quota (vCPU)	0.08	0.15 (compute-bound press) / 0.05 (mixed press) / 0.09 (data-access press)	0.08
Storage-member memory limit	512 MB	512 MB	512 MB
Clients per domain	24	24	24
MongoDB connections per edge server	6	12	6
Dynamic-unit cap (compute / storage)	3 / 3	6 / 6	12 / not exercised
Inter-domain link RTT (emulated)	185 ms	185 ms	185 ms
Service-path flow timeouts (idle / hard)	30 s / 60 s	30 s / 60 s	30 s / 120 s
Client bounds (in-flight window / request cap / phase-end drain)	1024 / 300 s / 30 s	1024 / 300 s / 30 s	1024 / 300 s / 30 s

Note. Two scaling-action runs reached the edge container's memory limit as the press began, at about 367–370 MB against the 384 MB limit of the mixed and data-access presses; one was replaced and one retained with its outcome flagged. In the readiness admission campaign storage scaling is not exercised, and the intact-source admission-timing sets use the higher quotas listed.

TABLE A.2. Scaling-policy parameters per campaign.

Parameter	Telemetry Modes (RQ1)	Bottleneck-Aware Scaling Action (RQ2)	Readiness Admission Modes (RQ3)
Latency signal statistic	mean	median	mean
Compute score weights (utilisation / latency)	0.60 / 0.40	0.60 / 0.40	0.40 / 0.60
Compute signal floors (CPU % / latency ms)	10 / 25	10 / 25	4.5 / 20
Compute scale-up threshold (base / max)	0.18 / 0.85	0.18 / 0.85	0.25 / 0.85
Storage score weights (utilisation / latency)	0.00 / 1.00	0.60 / 0.40	0.60 / 0.40
Storage signal floors (CPU % / latency ms)	1.5 / 60	10 / 10	1.5 / 60
Storage scale-up threshold (base / max)	0.35 / 0.55	0.35 / 0.55	0.20 / 0.55
Scale-up cooldowns (compute / storage)	45 s / 120 s	45 s / 120 s	45 s / 120 s
Compute scale-down (cooldown / window / below-criterion)	180 s / 3 of 6 / 25 % · 40 ms	180 s / 3 of 6 / 25 % · 40 ms	180 s / 7 of 12 / 15 % · 20 ms
Storage scale-down (cooldown / window)	30 s / 3 of 5	30 s / 3 of 5	120 s / 9 of 15
Action budget per tier	—	4	—
Bottleneck classification margin	—	0.05	—

Note. The latency statistic is the per-summary statistic that feeds the decision signals; the telemetry campaign and the readiness admission campaign keep the mean statistic of the platform base configuration, and the scaling action campaign runs the median statistic. In the telemetry modes column the zero utilisation weight makes the storage signal latency-only; the action budget and the classification margin apply only to the bottleneck-aware policy.

TABLE A.3. Load-balancing weights per campaign.

Parameter	Telemetry Modes (RQ1)	Bottleneck-Aware Scaling Action (RQ2)	Readiness Admission Modes (RQ3)
Backend weights (CPU / RAM / requests / hops)	0.30 / 0.10 / 0.20 / 0.28	0.30 / 0.10 / 0.20 / 0.28	0.20 / 0.20 / 0.20 / 0.28
Storage-member weights (CPU / RAM / connections / lag / hops)	0.20 / 0.10 / 0.20 / 0.20 / 0.30	0.20 / 0.10 / 0.20 / 0.20 / 0.30	0.20 / 0.20 / 0.10 / 0.20 / 0.30
Storage selection band (cost epsilon / minimum members)	—	3 % / 2	—

Note. Weights of the weighted cost functions that select a backend, and a storage member, for a new flow (Section 3.2.2). The readiness admission campaign pins these weights; the other two campaigns use the platform base weights. The weights need not sum to one, because each metric enters as a fraction of the largest value observed among the eligible candidates.

TABLE A.4. Control-plane cost per configuration (descriptive; medians across the runs of each configuration). Delivery records count delivered telemetry summaries; action records count provisioned units; admission records count events or readiness checks. The admission rows list the direct and discovery configurations; the fallback and confirmation checks are not tabulated.

Campaign	Configuration	Control records per run	CPU	Memory
Telemetry Modes (RQ1)	A push	340 deliveries; 6 compute scale-ups	14.3 %	69 MB
Telemetry Modes (RQ1)	B delayed push	332 deliveries; 6 compute scale-ups	14.7 %	69 MB
Telemetry Modes (RQ1)	C polling	105 deliveries; 4 compute scale-ups	14.5 %	72 MB
Telemetry Modes (RQ1)	D sampled push	114 deliveries; 3 compute scale-ups	14.0 %	73 MB
Bottleneck-Aware Scaling Action (RQ2)	no scaling, mixed press	none	4.5 %	77 MB
Bottleneck-Aware Scaling Action (RQ2)	compute-only, mixed press	3 compute scale-ups	3.7 %	73 MB
Bottleneck-Aware Scaling Action (RQ2)	storage-only, mixed press	7 storage admissions	5.1 %	68 MB
Bottleneck-Aware Scaling Action (RQ2)	bottleneck-aware, mixed press	7 storage admissions and 3 compute scale-ups	4.1 %	69 MB
Bottleneck-Aware Scaling Action (RQ2)	no scaling, data-access press	none	20.5 %	71 MB
Bottleneck-Aware Scaling Action (RQ2)	compute-only, data-access press	1 compute scale-up	22.1 %	70 MB
Bottleneck-Aware Scaling Action (RQ2)	storage-only, data-access press	6 storage admissions	21.1 %	75 MB
Bottleneck-Aware Scaling Action (RQ2)	bottleneck-aware, data-access press	6 storage admissions and 1 compute scale-up	24.2 %	75 MB
Readiness Admission Modes (RQ3)	backend notification	4 events	10.9 %	67 MB
Readiness Admission Modes (RQ3)	periodic polling, 10 s	4 check records	11.5 %	66 MB
Readiness Admission Modes (RQ3)	periodic polling, 15 s	4 check records	11.0 %	66 MB

Note. Controller CPU is the mean over the episode phase for delivery and action, and over the plateau for admission; memory is the corresponding mean. Action records count the scaling actions taken during the episode. Compute scale-ups are per domain. The telemetry campaign reports compute scale-ups per run, across both domains. Storage

admissions are activations of the prepared reserve per domain. They are not concurrent member counts. Counts are medians across that configuration's runs.

TABLE A.5. Implementation knobs of the interface experiments.

Knob	Meaning
READINESS_PROPAGATION	Compute-admission mode. off registers a replica immediately, direct admits on the app_ready message, and discovery checks on the polling interval.
READINESS_EVENT_FALLBACK_S	Direct mode. The time the gate waits for the message before checking anyway. 20 s in the reported admission runs and 130 s in the backend-notification configuration, where the check cannot begin before the abandonment bound.
DISCOVERY_POLL_INTERVAL_S	Polling interval. 10 s in the primary runs and 15 s in the sensitivity set.
READINESS_WAKE_VERIFY_MODE	probe enables the confirmation check on the direct base. The message wakes a check, and admission requires the check to pass.
READINESS_PROBE_TIMEOUT_S, _RETRY_S, _MAX_S	Check timeout (5 s), retry interval (1 s), and abandonment bound (120 s).
EDGE_READY_PREMATURE_S	Fault instrumentation. The replica reports readiness this many seconds before it can serve, ten seconds in the evaluated premature runs.
EDGE_READY_SEMANTIC_LIE_S	Fault instrumentation. Workload requests fail while the readiness predicate stays satisfied, for this window counted from bind (1200 s).
READINESS_EVENT_DROP_MODE	Fault instrumentation. The readiness message is dropped entirely.
TELEMETRY_SOURCE	Telemetry mode source. event_preserving implements push, delayed_event_preserving implements delayed push (DELAY_S, 30 s), poll implements polling (POLL_INTERVAL_S, 30 s in the campaign), and sampled_push implements sampled push (SAMPLE_EVERY, every third summary); zmq is the platform's raw push default.
STORAGE_POOL_RENEW_ENABLED, EDGE_POOL_RENEW_*	Storage read-path renewal. The controller broadcasts a renewal after a storage admission, and the edge rotates its read epoch in two waves, the second 45 s after the first and after the 30 s warm lease, with a 15 s minimum interval. Enabled in the scaling action campaign.
STORAGE_SELECTION_SPREAD_*	Storage selection band. Costs within 3 % of the lowest count as equal, and at least two members are kept. Enabled in the scaling action campaign and off (0 / 0) elsewhere.
VIP_FLOW_ISOLATION, EDGE_FLOW_ISOLATION	One fresh routing decision per measured request, with per-connection service-path flows.

TABLE A.6. The six request kinds that make up the workload mixes, and the tier each loads.

Request kind	What it does	Tier loaded
Content lookup	Reads a single content item (a point read).	Storage
Feed ranking	Performs a filtered item retrieval of candidate content, then ranks the candidates in the edge server.	Storage read and compute
Service pressure	Introspects the edge server's recent request activity.	Compute
Compute challenge	Solves a hash chain over a nonce in the edge server.	Compute
Content update	Writes an update to a content item.	Storage
Content aggregation	Computes a collection-level aggregation over content items.	Storage

APPENDIX B

Implementation Validation

Before any campaign, the platform was validated so that a difference observed later could be attributed to the interface under test rather than to the platform. The locked configuration was exercised end to end, each mechanism reproduced across replicates, and the telemetry path checked against its mode. Table B.1 states the behaviour each check establishes.

TABLE B.1. Behaviour established before the experiment campaigns, with the evidence of each check.

Property	Established behaviour, with its evidence
Integrated stability	Runs with every mechanism enabled stay inside the 3 % failure envelope (0.18 % and 0.41 %) with no tracebacks.
Run-to-run reproducibility	After the scale-down cooldown fix, replicates repeat their phase behaviour, leaving 0.23 % variability.
Member removal without disrupting established flows	Stale-rule failures during removal fall from 56–65 % to 1.4 %, and no epoch rotates, so a removal reaches only flows in flight.
Prepared storage reserve	An activated reserve carries data-path traffic while the set stays live.
Telemetry fidelity	Every produced summary is delivered or recorded as an aged-out gap, completeness and delay match the configured mode, and overhead stays bounded.

Note. Correctness is read from this behaviour rather than from code inspection. The mechanisms were exercised under the canonical workload and their effects reproduced across replicate runs. The readiness path is also exercised with fault instrumentation that can report readiness early, answer with errors while the predicate holds, or lose the message (Appendix A); the hotspot activation check (84.5 ms to 3.58 ms in both directions) validates the Tier 1 mechanism, which is disabled in the reported campaigns.

APPENDIX C

Deployment and Delivery Configuration

This appendix records the components as deployed and the implementation of the four telemetry modes, complementing Sections 4.1 and 4.4. Table C.1 lists the components of the testbed, and Table C.2 the source each telemetry mode selects.

TABLE C.1. Platform components as deployed.

Component	Implementation	Role
Domain network	Open vSwitch bridge per domain	Carries one domain's client, service, and data traffic
SDN controller (one per domain)	OS-Ken application (Python)	VIP interception, load balancing, flow installation, and the observation and action contexts
Telemetry aggregator (one per domain)	Python process on the ZeroMQ transport	Collects producer records and produces summaries
Edge server	Flask application on the Werkzeug server	Serves the workload, reports per-request telemetry, and exposes the readiness and drain endpoints
Storage member	`mongod` with a companion sidecar	Holds one replica-set member and reports replication and resource state
Inter-domain router	Linux container with traffic shaping	Joins the two domains and applies the emulated link profile
Workload driver	Python harness on the host	Issues the scheduled open-loop load from client namespaces

TABLE C.2. The four telemetry modes and their implemented sources.

Mode	Source	Implementation
push	`event_preserving`	ordered replay of the durable log, with each summary acknowledged
delayed push	`delayed_event_preserving`	the same replay released when each summary closes, plus a fixed 30 s delay, without burst replay
polling	`poll`	the controller retrieves only the most recent summary, every 30 s, so intermediate summaries are not delivered
sampled push	`sampled_push`	every third summary delivered as it is produced, with the intermediate ones dropped

APPENDIX D

Review Corpus

This appendix records the corpus behind Chapter 2: 35 works, the 27 that address an interface, listed in Table D.1, and the eight platform sources of Section 2.2, which are context rather than evidence of an RQ gap.

TABLE D.1. The 27 interface works, by interface. The middle column merges what a work varies with what it measures, and a work appears under its first interface.

Work	What it varies or contributes, and measures	What it leaves open
Telemetry (6)		
Yaseen [11]	Surveys counters telemetry in programmable networks	Visibility gaps named, not measured
Llorens-Carrodeguas [10]	SDN steering over a resized VNF pool; whole-system activation	Ready-to-traffic not isolated
Huang and Pierre [22]	Varies the scrape interval (5, 60 s, adaptive); scheduling accuracy	Late and missing updates not separated
Hung [23]	Collection and routing at one controller	The loop does not scale the service
Usman [24]	Survey of edge and container observability	Measurement interval left open
Zhou and Yong [25]	Monitors application errors instead of CPU load	Delivery of telemetry not varied
Scaling action (10)		
ElMrabet [26]	Taxonomy of horizontal, vertical and hybrid scaling	Tier receiving capacity not a dimension
Qu [9]	Autoscalers by indicator, estimation and timing	Kind of capacity an architectural given
Taherizadeh and Stankovski [27]	Edge taxonomy; adaptation intervals; multi-tier support	Tier policies noted, not varied
Toka [28]	Kubernetes HPA control model for edge clusters	Action is replica scaling only
Ju [29]	Adjusts replicas from a workload forecast	Forecast trigger; replica-only
Cheng [30]	Proactive autoscaling from a series forecast	Replica-only; no tier choice
Nikravesh [12]	Three-tier service; SLA violations vs. database capacity	No runtime choice between tiers
Dogani [31]	Autoscaling survey; resources may not add throughput	No mechanism to select the tier
Xie [32]	Locates the microservice whose degradation propagates	Bottleneck always a microservice
Li [33]	Compute instances and data replicas from a forecast; tenancy cost	Forecast trigger, not a live bottleneck
Load balancing and admission (5)		
Belgaum [34]	Survey of conventional and AI load balancing	Set of backends taken as given
Neghabi [35]	Static and dynamic mechanisms; lagging state	Admission step not isolated
Abdelltif [36]	Controller-hosted balancer over the OpenFlow pool	Membership change not measured
Bogdanov [1]	Storage balancer at second granularity; SLO violations	Ready-to-traffic interval not measured
Dreibholz and Mazumdar [13]	RSerPool registration, keep-alive, de-registration	Propagation delay left to policy
Orchestration and placement (6)		
Wang [37]; Peng [38]	SDN at the edge as architecture and proof of concept	Coordination inside a system not measured
Nain [39]	Survey of edge SDN optimisation challenges	Challenges named, not measured
Saurez [40]	Geo-distributed orchestration; staged deployment latency	No attribution to a single handover
Tamiru [41]	Geo-distributed Kubernetes orchestration	Coordination built in, not isolated
Tabatabaee Malazi and Clarke [42]	Dynamic service placement in MEC; launch to readiness	Readiness-to-traffic outside its scope